

Correspondence Matrices: A Structural Layer for Boolean Computation

Representing, transforming, decomposing, and routing Boolean logic — not replacing existing tools

Boolean computation has powerful tools for execution, canonicalization, and minimization. Correspondence Matrices introduce a structural representation layer that complements them.

STRUCTURAL BOOLEAN COMPUTING

OPERATOR CALCULUS

COMPILER ARCHITECTURE

Brian Theory

Boolean Computation Is Everywhere

It is not an old problem. It is a substrate of modern computation.



"Every domain that reasons about logic, rules, or structure touches Boolean computation. The tools are powerful — but fragmented."

The Boolean Tool Ecosystem Is Powerful but Fragmented

Each tool solves a different subproblem. None solves all of them.

Bitset / Numba

Fast flat truth-table execution. Best for one-shot packed output.

ROBDD / CUDD

Canonical decision-graph construction and equivalence. Extremely fast symbolic build. Variable-order sensitive.

SAT Solvers

Satisfiability and constraint solving. Best for feasibility queries, not structural analysis.

Espresso

Logic minimization and cover reduction. Best for compact representation.

SymPy

Broad symbolic algebra, parsing, rewriting, and CAS-style workflows.

Correspondence Matrices (CM)

Structural operator/matrix representation. Transformation, decomposition, measurement, reuse, and routing.



These methods are not competing to solve the same problem. The question is not which is fastest — it is which is right for the task.

What Is Actually Missing?

Most tools ask "how fast can I evaluate?" — but many workloads need more.

Existing tools answer well

- Evaluate all assignments → Bitset
- Canonicalize a function → ROBDD/CUDD
- Minimize logic → Espresso
- Solve constraints → SAT
- Manipulate symbolically → SymPy

The gap

No tool today primarily answers: What structure exists inside this Boolean computation? Can it be transformed, decomposed, measured, and reused?

Structural questions that matter

- Can this logic be transformed without re-evaluating?
- Can related expressions share compiled structure?
- Can structure guide which backend executes?
- Can operators be compared, decomposed, measured?

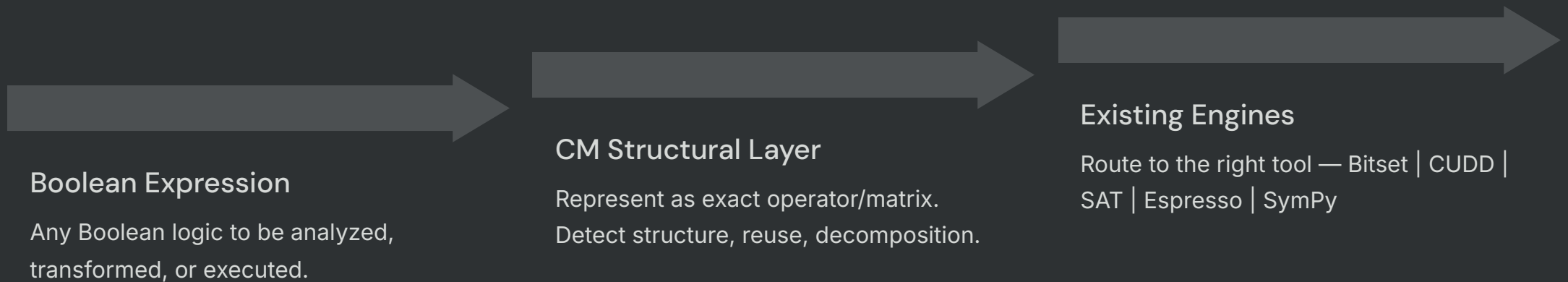
"The missing layer is structural intelligence — a representation that makes Boolean logic inspectable, transformable, reusable, and routable."



CM has not solved all of these problems yet; its operator/matrix formalism is a promising foundation.

The Structural-Layer Thesis

CM should not be judged only as another Boolean evaluator.



"CM should be evaluated as a candidate structural intermediate representation for Boolean computation — not merely as another evaluator."

✓ CM is the layer that represents and routes structure before execution. The engines below it remain the execution specialists.

What Is a Correspondence Matrix?

A Boolean operator represented as an ordered binary matrix — not just a truth table.

The familiar view

x	y	output
0	0	0
0	1	0
1	0	0
1	1	1

A truth table answers: what are the outputs?

The CM view

```
[0 0]  
[0 1]
```

A CM answers: what operator produced those outputs? How is it structured? Can it be transformed, decomposed, or reused?

- ① Under an ordered variable basis, every Boolean operator corresponds to a unique Correspondence Matrix. This makes Boolean logic an object — not just a function.

"CM gives Boolean logic an operator identity, not just an output."

Flat Output vs. Operator Identity

These are different questions. CM answers the second one.

Flat truth-table / Bitset view

Question answered: "What are the outputs?"

Output: A packed vector of 0s and 1s.

Best for: One-shot evaluation, fast execution, no structure needed.

Example: [0, 0, 0, 1] for AND(x,y)

CM operator/matrix view

Questions answered:

- What operator produced those outputs?
- How can it be transformed?
- Can it be decomposed into basis components?
- Can it be reused across related expressions?
- Can it be measured and reconstructed?

Output: A structured matrix identity under an ordered basis.

"Flat output is excellent for execution. CM is valuable when the structure of the operator matters — not just its outputs."

❗ "A truth table and a CM can represent the same Boolean function. The difference is what you can do with the representation afterward."

CM as Operator Calculus

Boolean logic becomes an object that can be operated on, not just evaluated.

Implemented / Experimentally Supported

- CM IR / DAG structural representation
- No-reinflate output path
- Structural-Hash Compiled-IR Cache
- Bitset backend delegation
- Live-variable reduction

Formalism-Supported / Partially Developed

- Matrix transformations (negate, transpose, rotate)
- Operator comparison
- Decomposition into basis components
- Measurement / reconstruction via bra/ket vectors
- Quotienting / feature difference

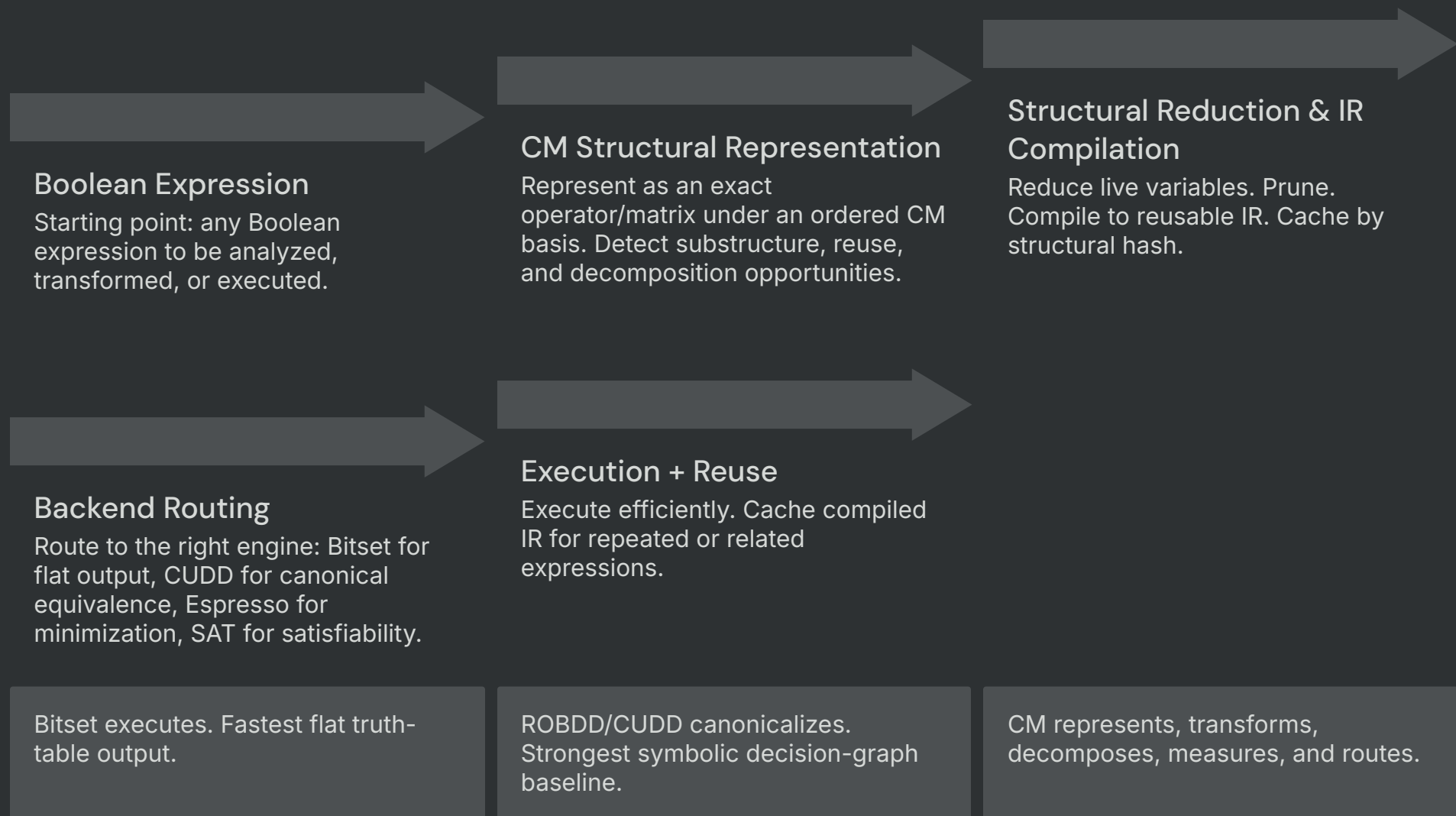
Future / Research Roadmap

- Canonical CM basis normalization
- Automatic backend routing
- Cost model
- Canonical CM structural equivalence
- Larger-n validation
- Real-workload validation

⚠ Not all capabilities are fully productized. Some are formal properties of the CM formalism, some are implemented, and some are roadmap targets. This distinction matters for credibility.

CM Represents. Bitset Executes. CUDD Canonicalizes.

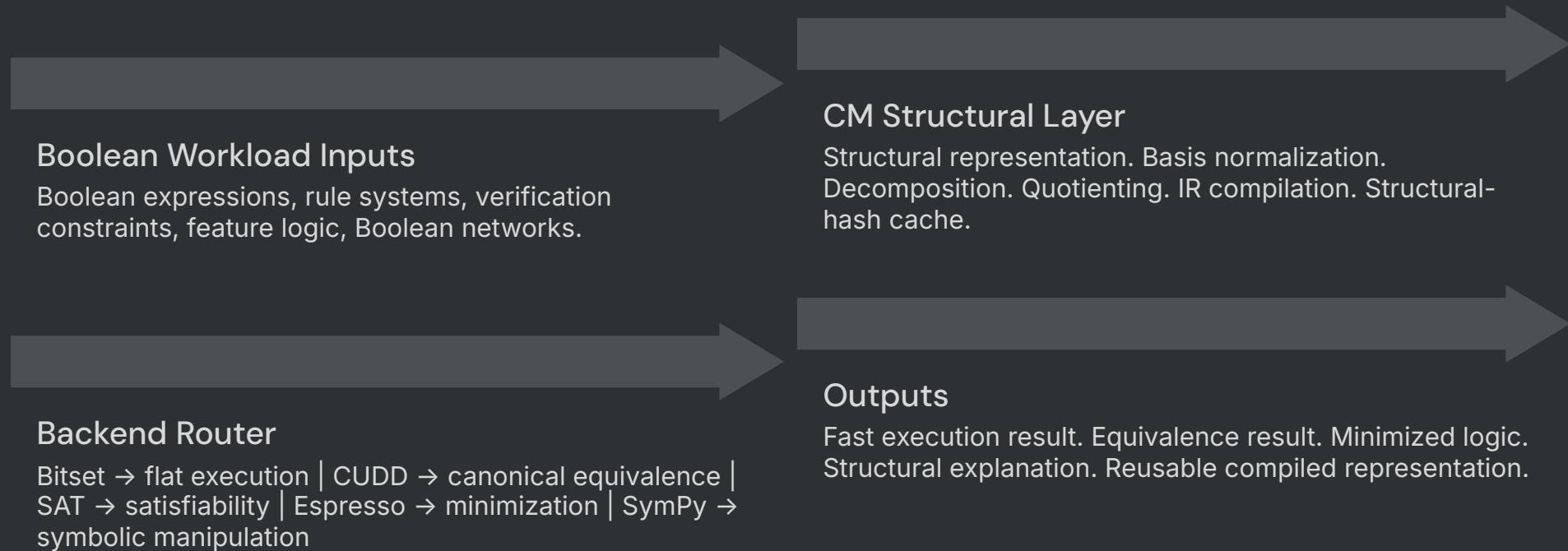
The honest thesis — CM complements the Boolean ecosystem, it does not replace it.



"CM does not compete with the Boolean toolbox. It gives the toolbox a structural brain."

A Structural Intelligence Layer for Boolean Workloads

The product is not a faster Boolean evaluator — it is a Boolean computation router/compiler.

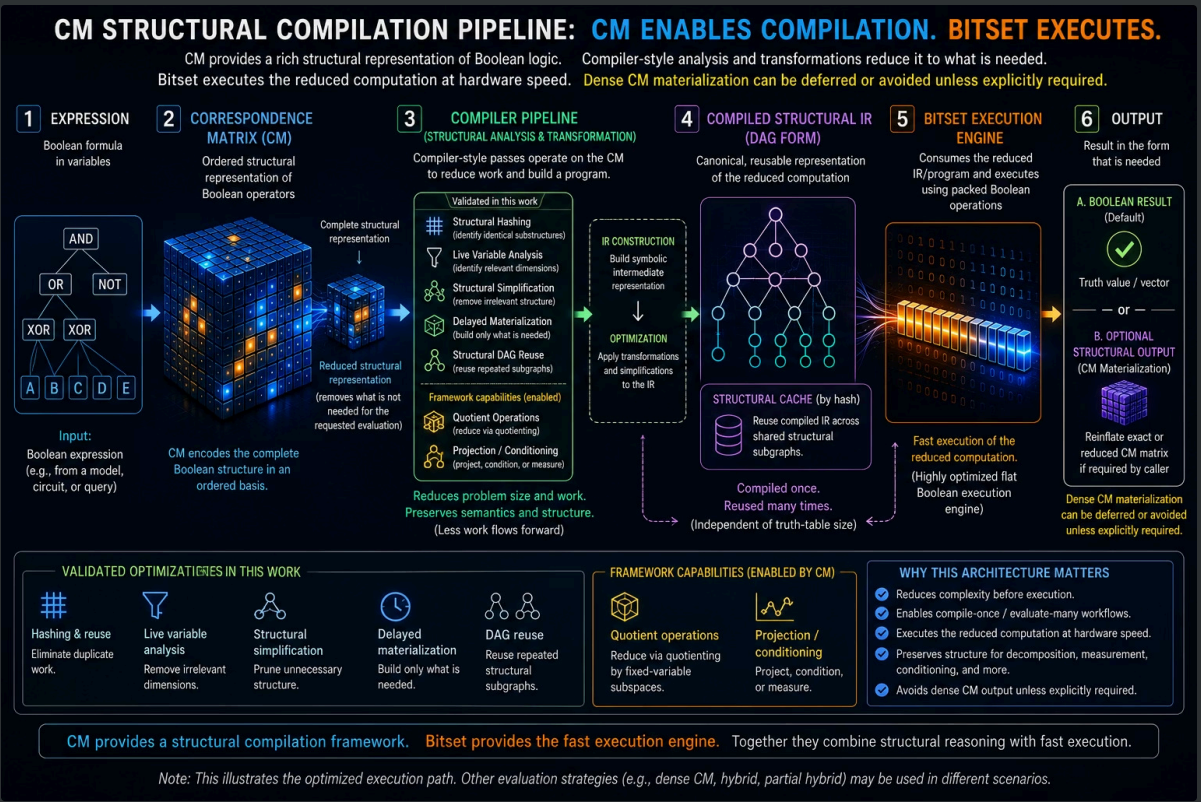


✓ The product is not 'faster Bitset.' It is a Boolean computation router/compiler built around structural representation.

i This is a research prototype vision. The current implementation validates the CM structural layer and no reinflate/structural-hash cache architecture. Backend routing and cost model are future work.

CM Structural Compilation Pipeline

CM provides structural representation; compiler passes reduce and cache; bitset executes the reduced computation.



Note: "hardware speed" here means fast packed Boolean operations, not a custom hardware implementation.

The Experiments Did Not Merely Confirm CM — They Refined It

Negative results were as important as positive ones.

What we tested

1. Can dense CM beat Bitset directly? → No.
2. Does CM_parallel provide scalability? → No — structural reduction removed the dense work.
3. Does preserving more structure automatically help? → No — a cost model is needed.
4. Does boundary optimization close the gap? → Partially — but not the dominant bottleneck.
5. Does no-reinflate help? → Yes — major improvement.
6. Does IR caching help? → Yes — compile-once/evaluate-many is the right architecture.
7. Does CUDD change the flat-output story? → No — CUDD build+extract remains far slower than Bitset.

What we learned

- Dense CM is not the right default output path.
- CM_parallel is not the scalability story.
- Structural reduction is the key — not multiprocessing.
- No-reinflate avoids the dominant output cost.
- IR compilation should be amortized, not paid per evaluation.
- CUDD is the strongest ROBDD baseline for symbolic construction.
- The right thesis is: CM as a structural compiler/IR layer, not CM as a raw evaluator.

"The value of CM became clearer precisely because several naive implementations were not the winning path."

Five Results That Changed the Thesis

The evidence supports CM as a structural compiler/IR layer — not a universal raw-speed evaluator.



Dense CM Did Not Beat Bitset for Flat Execution

This invalidated the simplistic thesis. If the only desired artifact is a flat truth-table output, Bitset is the correct first choice.



No-Reinflate Sharply Reduced CM Overhead

Avoiding dense final matrix output was one of the strongest improvements. At $n=16$ (threshold-9), no-reinflate reached $1.69\times$ of Bitset — the strongest one-shot CM result.



IR Compilation Became the Dominant Reusable Cost

After no-reinflate, IR compilation dominated. This supports the compiler interpretation: compile cost should be amortized, not paid per evaluation.



Structural-Hash Caching Improved Repeated Workflows

$1.51\times$ – $1.89\times$ end-to-end speedups. Cached CM reached $2.42\times$ of cached Bitset at $n=16$ while retaining CM structure. Matters only when structure repeats.



CUDD Showed Symbolic Build Is a Different Task from Flat Output

CUDD symbolic build: 0.000138 s at $n=16$. CUDD build+extract: 0.513589 s. These are different tasks. CUDD is the strongest canonical symbolic baseline.

"The evidence supports CM as a structural compiler/IR layer, not a universal raw-speed evaluator."

Experimental Roadmap: How the Architecture Emerged

The final CM architecture emerged through a sequence of positive and negative experiments.



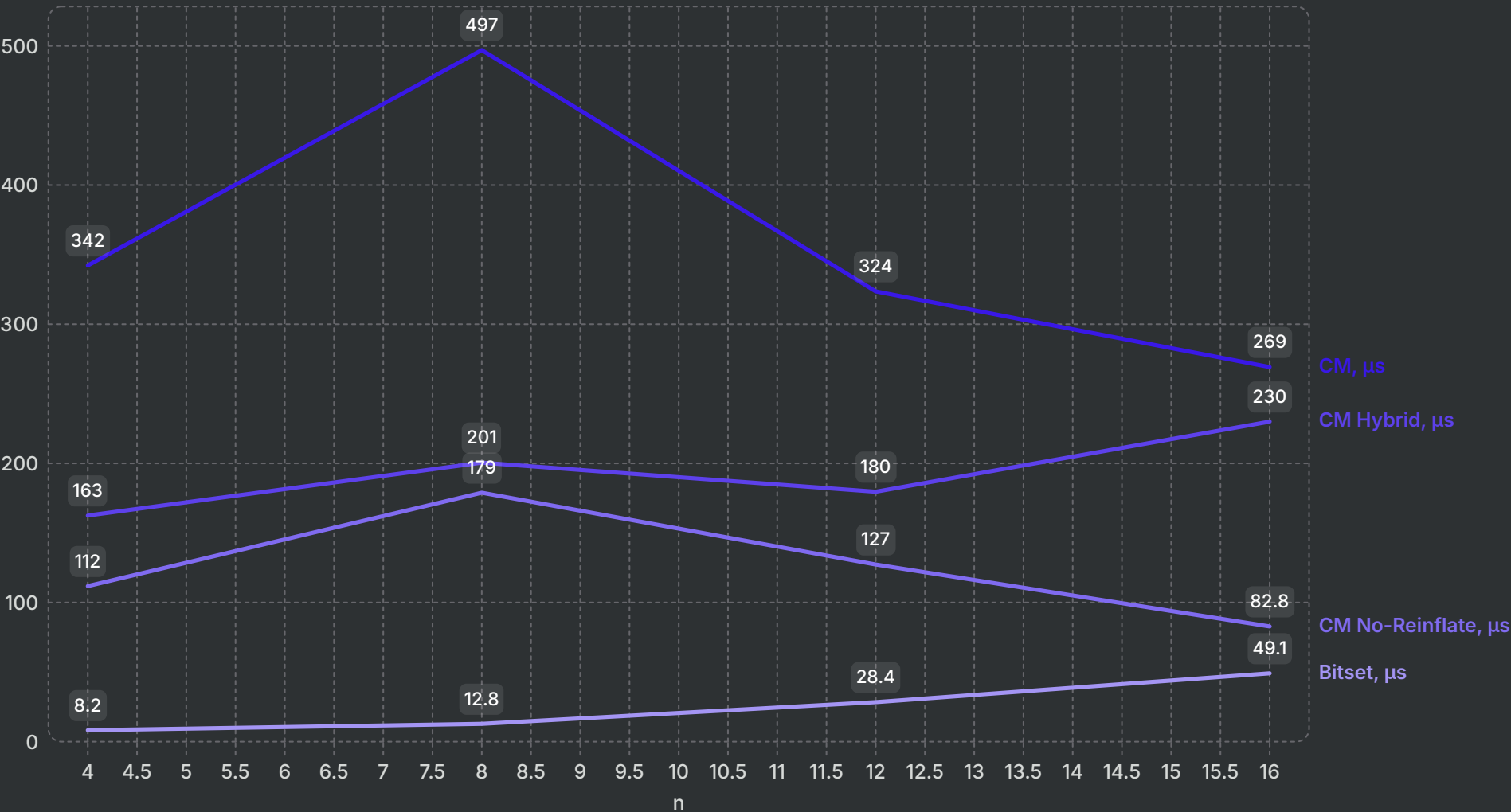
"The project evolved from 'Can dense CM beat bitset?' to 'Can CM reduce, preserve, and reuse structure before delegating execution?'"

Act III — What the Experiments Show

No-Reinflate: Best Current CM Flat-Output Path

No-reinflation avoids converting the final result back into a dense 2D CM matrix unless that matrix is actually required. This was one of the strongest improvements in the project — demonstrating that representation/output cost can matter as much as computation cost in this benchmark setting.

Threshold = 9 Results



n=16: CM No-Reinflate 82.8 µs vs Bitset 49.1 µs = 1.69x — the strongest one-shot CM execution result in the deck.

**CM Architectural Progress**

CM No-reinflat avoids dense CM matrix materialization when the dense matrix itself is not required. The dominant cost shifted away from dense matrix materialization and toward compilation, preprocessing, and execution orchestration.

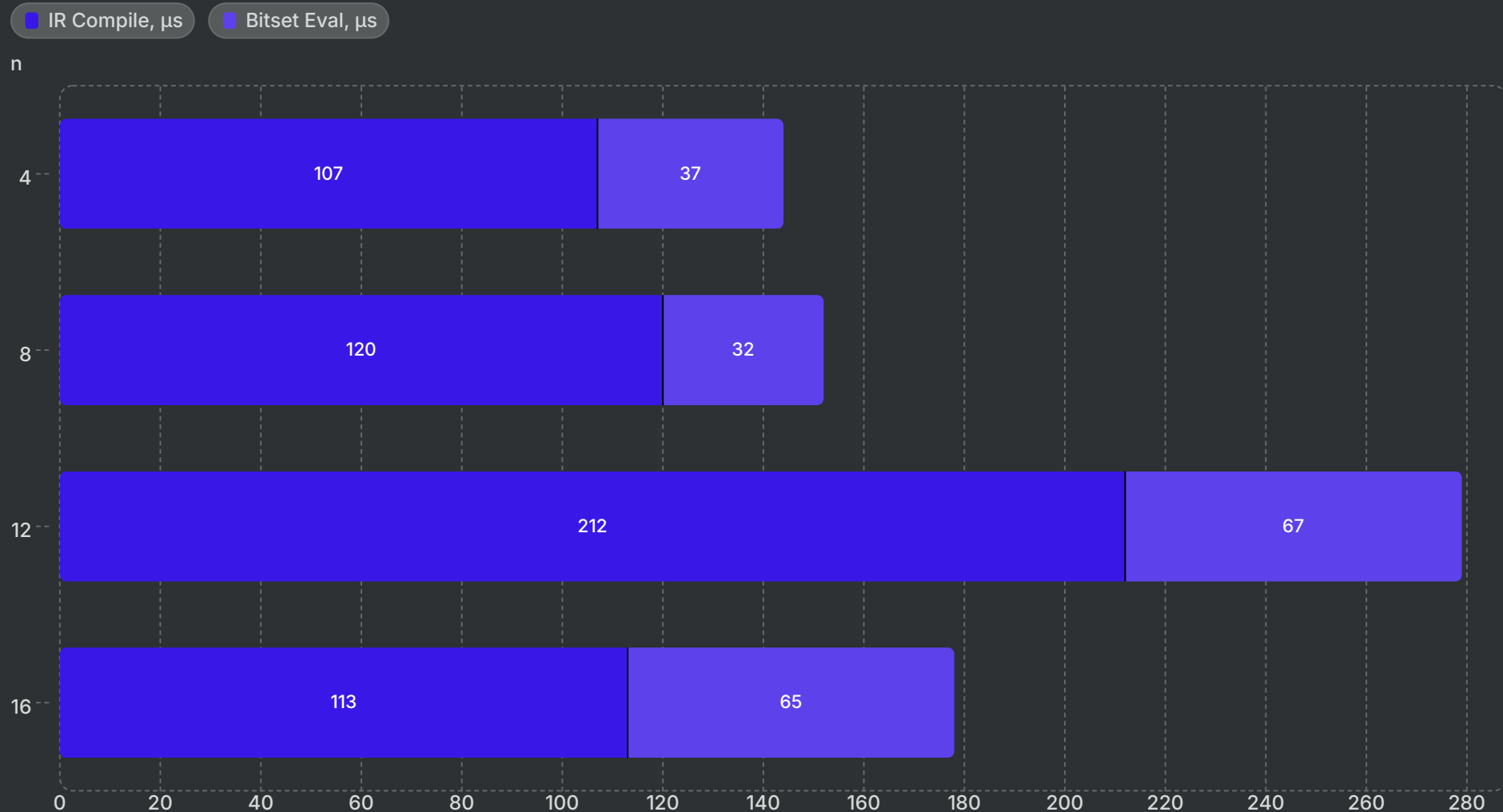


Favorable threshold-9 regime. Larger-n validation is required before claiming asymptotic convergence.

Act III — What the Experiments Show

IR Cost Decomposition

After no-reinflation, the remaining bottleneck was not bitset execution — it was CM IR compilation and execution scaffolding. This table decomposes where no-reinflat time is actually spent.



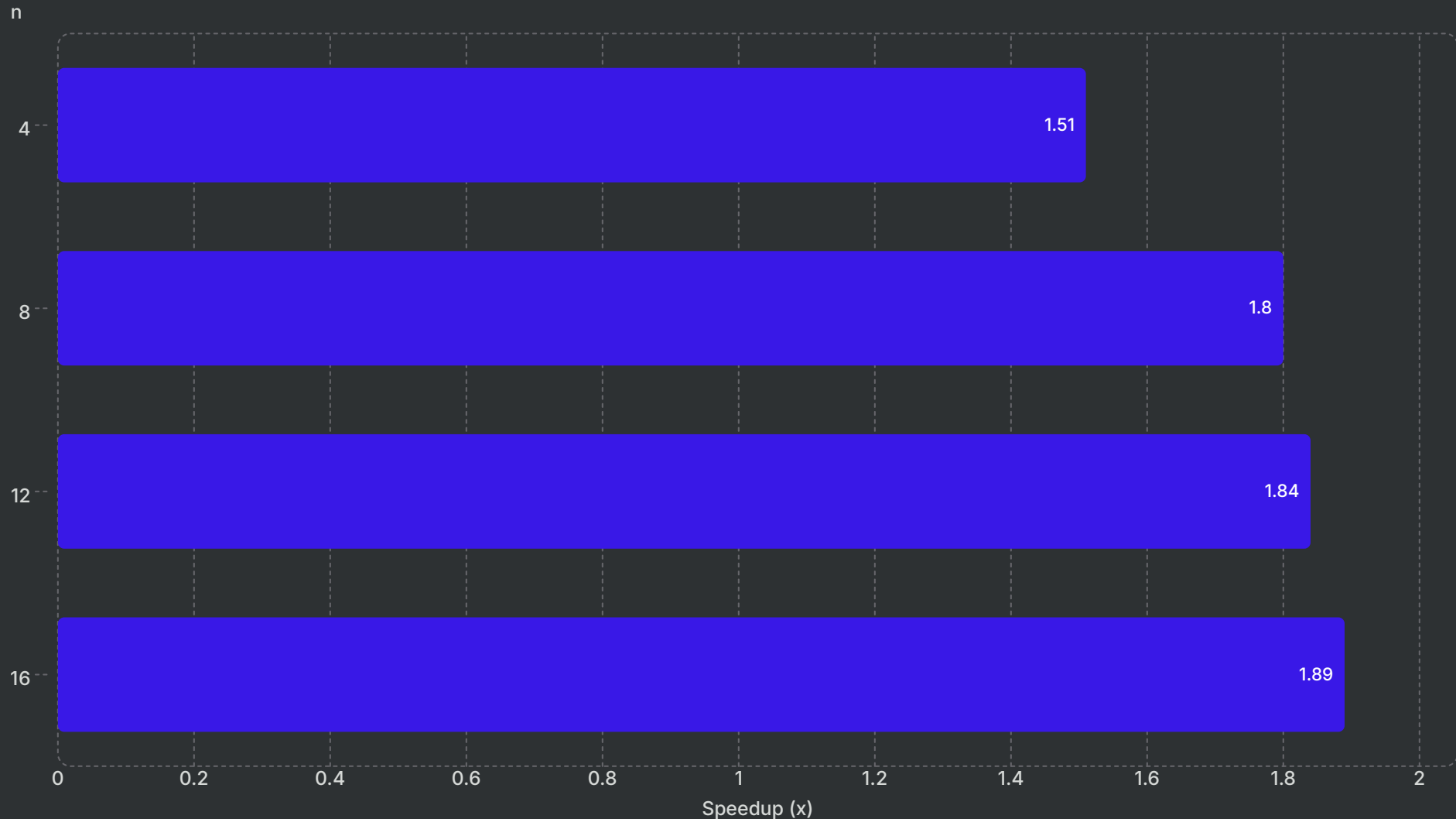
IR compilation dominates the remaining cost after no-reinflat. This supports the compiler interpretation: CM compile cost should be amortized or cached rather than paid for every one-shot flat evaluation.

Detailed intern/canonicalize/rewrite/live-vars columns remain in the appendix.

Act III — What the Experiments Show

Structural-Hash Cache: End-to-End Speedups

Structural-hash caching extends IR reuse across repeated evaluations of structurally identical expressions within the runtime.

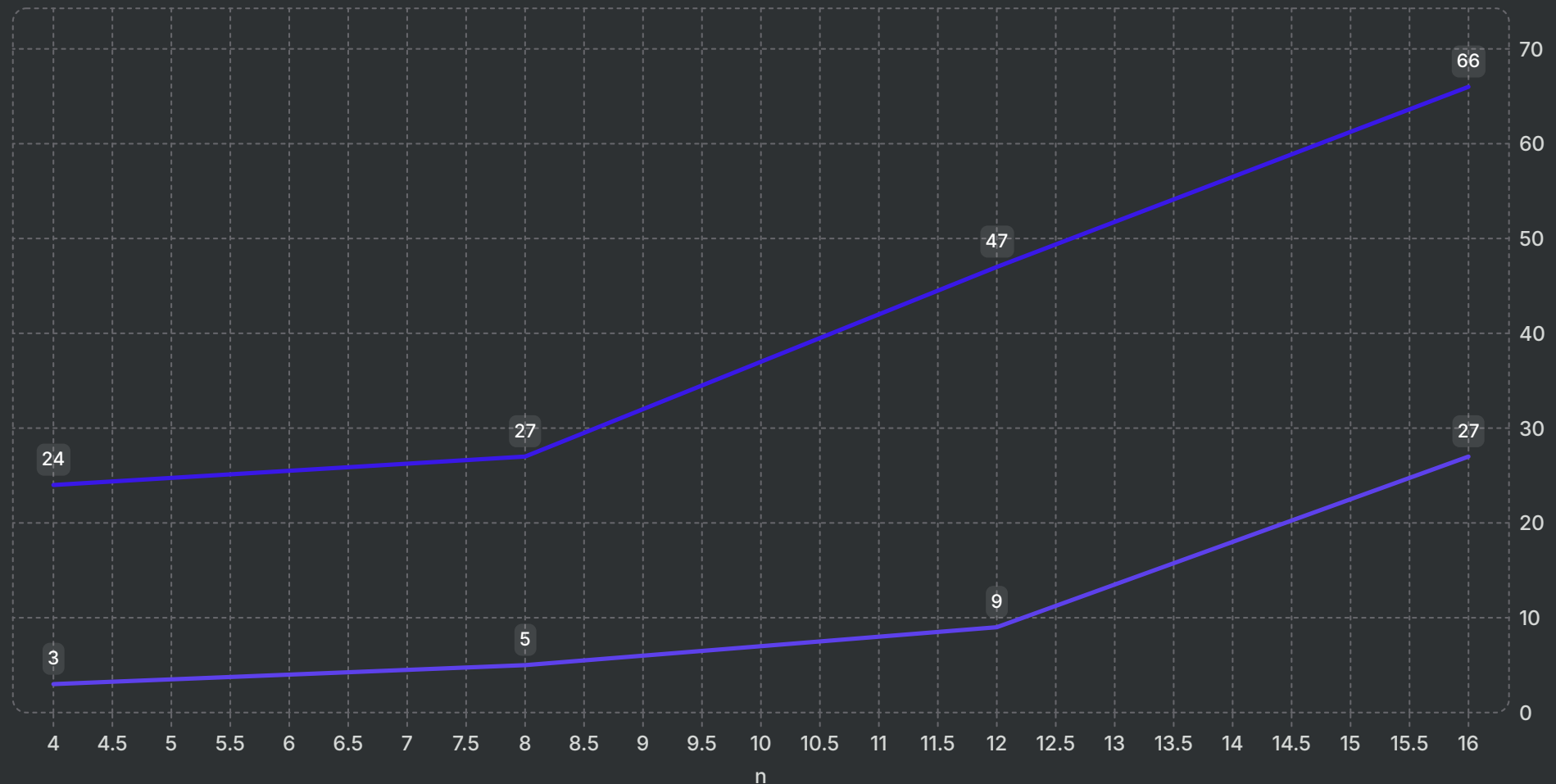


Structural-Hash Compiled-IR Cache reduces repeated compilation work and turns CM IR into a reusable compile artifact within the runtime.

Cached CM Narrows the Gap, but Bitset Still Wins Flat Execution

Cached CM remains slower than cached bitset for flat execution, but the gap narrows as n increases.

— Cached Bitset, μs — Cached CM, μs



i Cached CM remains slower than cached bitset for flat execution, but the gap narrows to 2.42x at $n=16$ while retaining CM structure.

⚠ This is not a claim that cached CM beats cached bitset. It is a claim that reuse makes CM more competitive when structure matters.

CUDD Changes the Symbolic Story — Not the Flat-Output Story

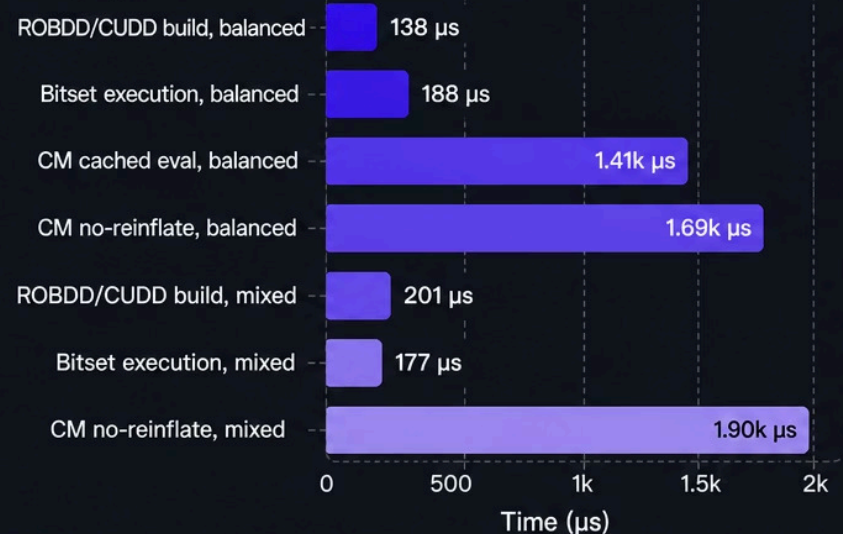
CUDD is the strongest ROBDD baseline. It changes the symbolic-build comparison substantially. It does not change the flat-output conclusion.

CUDD is the strongest ROBDD baseline. It changes the symbolic-build comparison substantially. It does not change the flat-output conclusion.

Symbolic Build and Fast Flat Timings, n=16

Time (μ s)

Category



CUDD symbolic build is extremely fast: faster than CM no-reinflate and near bitset timing in these Docker/Linux runs. CUDD build / CM no-reinflate ratio: **0.08x** (balanced) and **0.11x** (mixed) — CUDD builds the decision graph in roughly 1/10th the time of CM no-reinflate.

Build + Truth-Table Extraction, n=16

Time (ms)

Category



Flat-output methods are included for scale. Bitset (**0.000188 ms**) and CM no-reinflate (**0.001688 ms**) remain visible only as labeled values because truth-table extraction is approximately five to six orders of magnitude slower (**513–1130 ms**). This comparison illustrates that extraction—not symbolic graph construction—dominates flat-output workflows.



Environment note: CUDD results are from Docker/Linux (WSL2, python:3.10-slim, dd 0.5.7, dd.cudd import succeeded). Native Windows could not import dd.cudd. **dd.autoref** results are from the native Windows environment. Cross-environment comparisons should be interpreted with care.



Environment note: CUDD results are from Docker/Linux (WSL2, python:3.10-slim, dd 0.5.7, dd.cudd import succeeded). Native Windows could not import dd.cudd. **dd.autoref** results are from the native Windows environment. Cross-environment comparisons should be interpreted with care.

Graph Summary: The Speed Story

Bitset remains fastest for flat execution, but no-reinflat CM narrows the gap as structure and reuse become more important.

What the Timing Graphs Show

- 1

Bitset remains fastest for one-shot flat output. Best when the only goal is truth-table-style evaluation.
- 2

Hybrid CM improves over NumPy CM but does not beat bitset for flat output.
- 3

No-reinflat removes dense output cost and sharply improves CM's flat-output path.
- 4

IR compilation becomes the remaining cost after no-reinflat.
- 5

Structural-Hash Compiled-IR Caching improves repeated or related workflows — compile-once / evaluate-many.
- 6

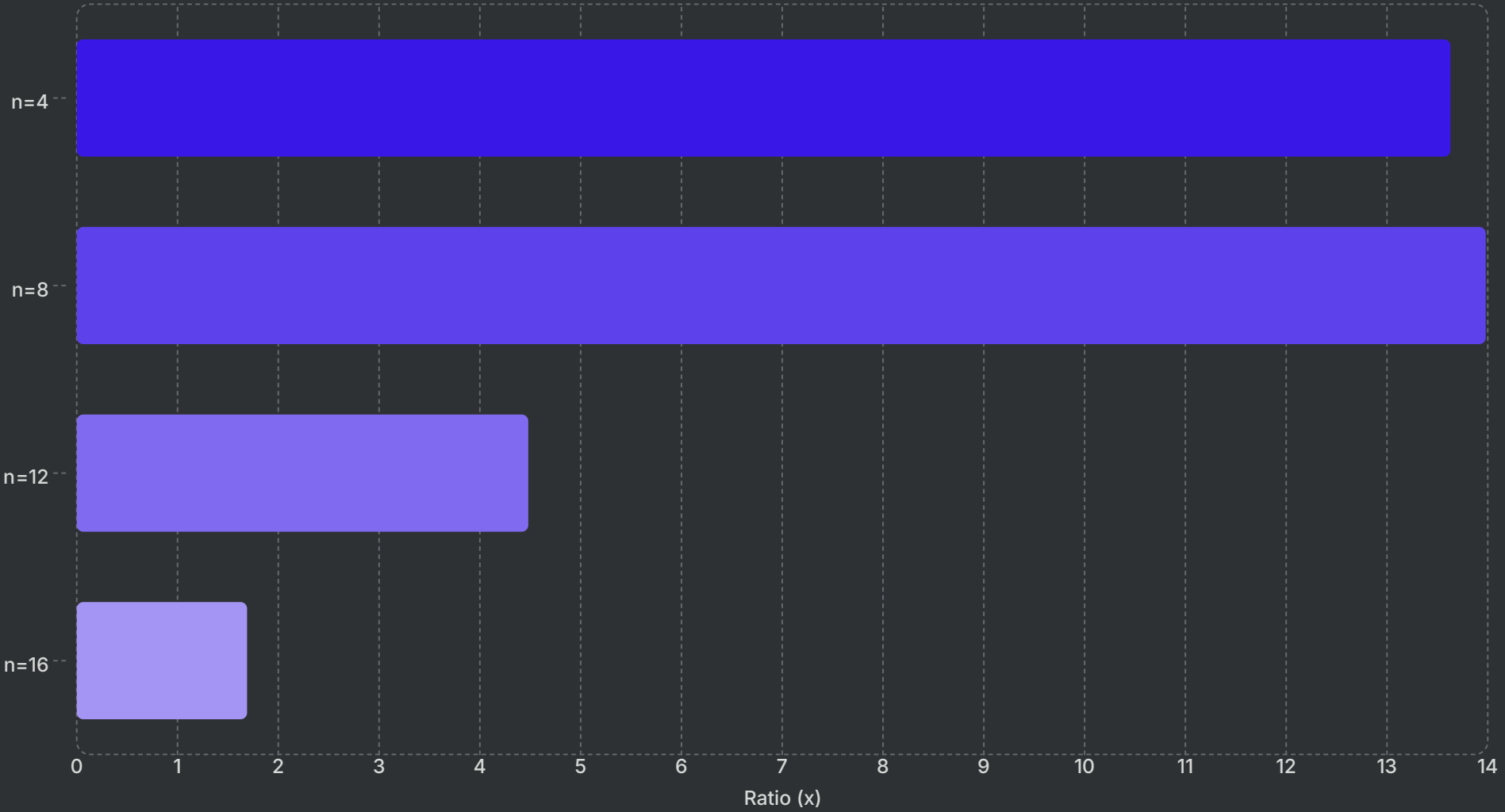
CUDD/ROBDD is extremely strong for symbolic build and canonical graph construction, but build-only is not the same as flat truth-table extraction.

Most Important Numeric Trend

No-Reinflat / Bitset Ratio, Threshold 9

Ratio, x — lower is better

n value



No-reinflat narrows the gap to bitset in the favorable threshold-9 n=12 to n=16 regime, reaching 1.69x at n=16. This suggests a possible trend, but larger-n validation is required before claiming asymptotic convergence.

This is a favorable threshold-9 run. Harder all-vars/mixed CUDD comparison regimes show CM no-reinflat around 9–11x slower than Bitset at n=16 for flat output. This slide should not imply CM is converging universally toward Bitset.

Bitset answers quickly. CM explains, reduces, transforms, and reuses structure before handing execution to bitset.

What the Evidence Supports

The strongest claims are architectural, not universal raw-speed claims.

Supported Now

- CM as structural representation
- No-reinflate as the best current CM output path
- Structural-hash compiled-IR cache improves repeated workflows
- Bitset remains fastest for flat output
- CUDD/ROBDD is the strongest symbolic/canonical decision-graph baseline

Promising but Not Fully Proven

- Larger-n scaling
- Related-expression family wins
- Partial evaluation / conditioning workload wins
- Automatic backend routing
- Cost-model-driven method selection
- Canonical CM structural equivalence

"The strongest current thesis: CM represents, reduces, and reuses Boolean structure; bitset executes flat output; CUDD canonicalizes symbolic decision graphs."

Final Architecture Thesis

CM is not 'faster bitset.' CM is a structural layer that helps decide what should be represented, reduced, reused, or routed before execution.

CM

Represents Boolean logic as structured operator/matrix objects. Supports transformation, decomposition, quotienting, measurement, conditioning, reuse, and routing.

Compiler Pipeline

Analyzes CM structure, reduces live variables, builds symbolic IR, avoids dense materialization when unnecessary, and caches compiled structure via Structural-Hash Compiled-IR Cache.

Bitset

Executes the reduced flat Boolean computation efficiently. Fastest for one-shot flat output.

CUDD / ROBDD

Canonicalizes symbolic decision graphs. Strongest mature baseline for symbolic equivalence workflows. Extremely fast symbolic build.

Correspondence Matrices provide a structural layer for Boolean computation: they make Boolean logic inspectable, transformable, decomposable, measurable, reusable, and routable before execution.



Architecture pattern: Compile structure once → reuse compiled IR when structure repeats → execute reduced subproblem with bitset → avoid dense CM reinflation unless the CM matrix itself is required.

Next Experiments

The next question is not whether CM beats every tool. It is where structural representation changes the cost model of Boolean computation.

- 1 Larger-n validation beyond $n=16$
- 2 Repeated related-expression families
- 3 Partial evaluation and conditioning workloads
- 4 Canonical CM basis normalization
- 5 CM structural equivalence benchmarks
- 6 Cost model for backend routing
- 7 Fair CUDD equivalence comparison in Docker/Linux
- 8 Real-world Boolean workloads

Answering even two or three of these would substantially test the structural-layer thesis and clarify where CM changes the cost model of Boolean computation.

First priority: benchmark CUDD canonical equivalence in Docker/Linux. This is the most immediate open question.

APPENDIX

Appendix

Detailed benchmarks, method comparisons, caveats, and backup evidence.

Each Method Optimizes a Different Thing

The goal of this analysis is not to rank methods by raw speed alone. Different methods solve fundamentally different subproblems in Boolean computation.

Bitset

Optimizes raw flat execution over packed truth assignments

ROBDD/CUDD

Optimizes canonical reduced decision-graph representation under a variable ordering. CUDD symbolic build is extremely fast — competitive with bitset in Docker/Linux runs. Does not primarily optimize full truth-table extraction; build + extraction remains far slower than bitset.

SymPy

Optimizes broad CAS-style symbolic expression manipulation, simplification utilities, and readable algebraic workflows

Espresso

Optimizes logic minimization

Numba

Optimizes compiled evaluator execution

CM

Optimizes **Boolean operator representation, matrix-based transformation, decomposition, quotienting, measurement, reusable structure, and problem reduction**



Strongest current interpretation: CM is most beneficial as a Boolean operator calculus and structure-preserving compiler/IR. Bitset is the strongest flat execution kernel. ROBDD/CUDD is the strongest mature canonical decision-graph baseline — extremely fast for symbolic construction, but build + truth-table extraction is a different and much slower task. SymPy is strongest for broad symbolic algebra tooling. The most promising CM architecture is: CM operator representation → structural reduction → bitset execution → no-reinflate → Structural-Hash Compiled-IR Cache.



Important: ROBDD/CUDD build-only should not be compared directly to bitset flat output unless truth-table extraction is included. These are different tasks.

High-Level Method Comparison

A qualitative capability map. Stars summarize the reported benchmark regime and architectural fit; they are not universal rankings.

Methods grouped by role — Execution kernels: Bitset, Cached bitset, Numba | **Symbolic/minimization baselines:** ROBDD/CUDD, SymPy, Espresso | **CM family:** Dense CM, Balanced/Symbolic CM IR, CM hybrid, CM no-reinflate, CM no-reinflate + Structural-Hash Compiled-IR Cache

Method	Speed	Scalability	Repr.	Reuse	Best Use
Bitset	5	2	1	2	Fast flat truth-table evaluation
Cached bitset	5	2	1	3	Repeated flat evaluation with no structure needed
Numba	3	2	1	2	Compiled evaluator baseline
ROBDD/CUDD	2–5	2–5	5	5	Canonical decision-graph construction and equivalence. CUDD symbolic build is extremely fast; build + truth-table extraction is slow for flat output.
SymPy	2	2	4	4	Broad symbolic algebra tooling
Espresso	2–3	3	2	3	Logic minimization
Dense CM / NumPy CM	3	3	5	4	Exact CM matrix / operator representation
Balanced / Symbolic CM IR	3–4	4	5	4	Symbolic CM IR / DAG reduction before execution
CM hybrid	4	4	5	4	CM structure plus bitset execution
CM no-reinflate	4	4–5	5	4	Best CM path without dense output
CM no-reinflate + Structural-Hash Compiled-IR Cache	4	5	5	5	Compile-once / evaluate-many with CM structure

Speed and scalability ratings reflect the reported benchmark regime and architectural analysis. ROBDD/CUDD is shown with ranges because performance can be excellent under favorable variable ordering but can blow up under unfavorable ordering or random-expression regimes.

ROBDD/CUDD speed depends on the task: symbolic build is excellent (competitive with Bitset at n=16); build+truth-table extraction is slow for flat output (2727x slower than Bitset at n=16, balanced_all_vars).

i

Key takeaway: Bitset wins flat output. ROBDD/CUDD wins mature canonical decision-graph construction and equivalence workflows. CMs earn top representation scores because ordered CMs provide exact operator/matrix representations of Boolean logic. Note: ROBDD/CUDD speed depends on the task — symbolic build is excellent; build+truth-table extraction is slow for flat output.

CORRESPONDENCE MATRICES (CMs)

A Structural Representation for Boolean Computation

BITSET

Bit-parallel truth-table evaluation
Packed bits for fast execution

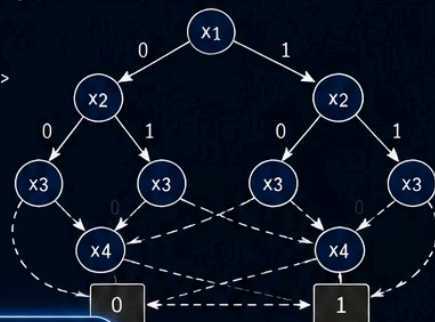
Assignments (rows)	Word 0 (SIMD / CPU word)	Word 1 (SIMD / CPU word)
0	0 1 1 0 1 0 1 0	1 0 1 1 0 1 0 1 ...
1	1 0 1 1 1 0 1 1	0 0 1 1 0 1 1 0 ...
2	1 0 1 0 1 1 0 0	0 1 0 0 1 1 0 0 ...
3	1 1 0 1 1 0 0 1	1 1 0 1 1 0 0 0 ...
⋮	⋮	⋮
$2^N - 1$	1 0 1 0 1 1 1 1	1 0 1 1 0 1 1 1 ...

Exact operator representation that unifies
symbolic structure and high-performance execution

ROBDD (BDD)

Reduced Ordered Binary Decision Diagram
Shared subgraphs enable compression

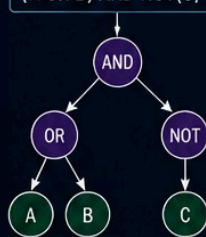
Variable
order:
 $x_1 > x_2 >$
 $x_3 > x_4$



SYMPY

Symbolic algebra
Expression manipulation
Algebraic rewriting

(A OR B) AND NOT(C)



CORRESPONDENCE MATRICES (CMs)

Exact Boolean operator representation
Structured operator correspondence

Op.	Matrix	Op.	Matrix	Op.	Matrix	Op.	Matrix
$[\Phi]$	$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$	$[0]$	$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$	$[\Leftrightarrow]$	$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	$[0]$	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
$[\wedge]$	$\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$	$[\neg]$	$\begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$	$[t]$	$\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$	$[U]$	$\begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$
$[\Rightarrow]$	$\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$	$[\Leftarrow]$	$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$	$[V]$	$\begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}$	$[\neg A]$	$\begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$
$[R]$	$\begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix}$	$[\neg R]$	$\begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$	$[L]$	$\begin{bmatrix} 1 & 1 \\ 0 & 0 \end{bmatrix}$	$[\neg L]$	$\begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$

Basis states (for 1 Boolean variable)

$$\begin{aligned} |1\rangle \times |1\rangle &= \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, & |1\rangle \times |0\rangle &= \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \\ |0\rangle \times |1\rangle &= \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, & |0\rangle \times |0\rangle &= \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned}$$

ESPRESSO

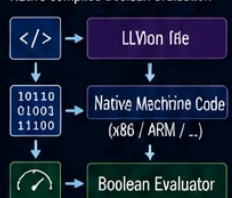
Heuristic Boolean minimization
Two-level logic minimization

1) Cube Covers 2) Prime Implicants 3) Two-Level Logic Minimization



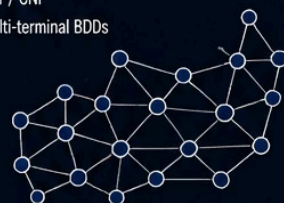
COMPILED EXECUTION (Numba / LLVM)

JIT / AOT compiled evaluators
Native compiled Boolean evaluation

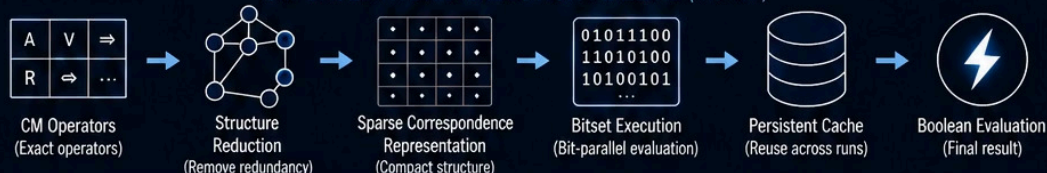


OTHER METHODS

- Explicit Truth Tables
- Decision Tables / Rule Tables
- SAT / CNF
- Multi-terminal BDDs
- ...



CM-BASED HYBRID EXECUTION PIPELINE (This Work)



Different representations. Different strengths. Correspondence Matrices expose operator-level structure that supports hybrid symbolic and bit-parallel execution.

Appendix — Section 1 (continued)

How to Read the Ratings

Four important caveats behind the qualitative capability map.

Ratings Are Qualitative

Stars summarize the reported benchmark regime and architectural fit. They are not universal performance laws.

ROBDD/CUDD Range

ROBDD/CUDD can be extremely fast for symbolic construction and canonical reasoning, especially with favorable variable ordering. But if the desired output is a full truth table, build + extraction is a different and much slower task.

CM Memory

CM cells are binary, but dense CMs can still require exponentially many cells. Memory ratings reflect total representation footprint, not per-cell complexity.

CM Scalability

CM scalability comes from symbolic IR, live-variable reduction, no-reinflate output, and structural-hash compiled-IR caching — not from multiprocessing.

Terminology note: older figure labels such as "persistent cache" correspond to the Structural-Hash Compiled-IR Cache terminology used in this deck.

What Each Method Is Really Optimizing

This conceptual table prevents unfair comparisons by clarifying that methods are solving fundamentally different subproblems. Understanding these distinctions is essential to interpreting the benchmark results correctly.

Method	What It Primarily Optimizes	What It Does Not Primarily Optimize
Bitset	Fast packed execution over many truth assignments	Symbolic structure, decomposition, interpretability
Numba	Compiled execution of evaluator logic	Reusable symbolic / algebraic representation
ROBDD/CUDD	Canonical symbolic decision-graph construction and equivalence under a variable ordering. CUDD symbolic build is extremely fast.	Full truth-table extraction or flat packed-output generation. Build + truth-table extraction is far slower than Bitset for flat output.
SymPy	Broad general-purpose symbolic expression manipulation, simplification utilities, and CAS-style workflows	Low-level packed truth-table execution
Espresso	Boolean logic minimization	Matrix/operator decomposition or repeated execution
Dense CM	Structured matrix/operator representation	Raw packed execution speed
CM hybrid	Structure-preserving reduction plus bitset execution	Eliminating all CM/bitset boundary overhead
CM no-reinflate	Structure plus execution without final dense CM matrix output	Eliminating all remaining IR traversal, dispatch, and wrapper overhead
CM + structural-hash compiled-IR cache	Reusable compiled structure and amortized execution	Being faster than bitset for every one-shot flat evaluation

✔ **Strongest project result:** CM creates reusable operator/IR structure; bitset executes the reduced form.

📖 CM is not merely "structure preserving." In the original formalism, CMs form a matrix calculus for propositional logic: operators can be represented as binary matrices, transformed by matrix operations, decomposed into basis matrices, quotiented, and measured by logical bra / ket vectors. This formal CM strength is distinct from the implementation benchmark strength, which comes from symbolic IR, no-reinflate output, bitset execution, and Structural-Hash Compiled-IR Cache.

Representation and Structure Preservation

Structure preservation determines whether a method retains useful information about *how* a Boolean computation is organized — enabling reuse, decomposition, conditioning, and interpretability beyond a raw truth table.

Method	Representation	Reuse	Interpretability	Decomposition	Symbolic Build Speed	Flat Output Speed
Bitset	1	2	1	1	1	5
ROBDD/CUDD	5	5	4	4	5	1
SymPy	4	4	5	3	2	2
Dense CM	5	4	5	5	2	2
CM No-Reinflate	5	4	5	5	3	4

Scale: 1 = weak, 5 = strong. Higher is better for all dimensions. 'Symbolic Build Speed' = speed of constructing the representation. 'Flat Output Speed' = speed of producing a full truth-table-style output vector.

The heatmap compares five capabilities on a 1–5 scale. Bitset emphasizes execution speed but preserves little structure, while ROBDD/CUDD, SymPy, and CM variants retain more representation and reuse benefits. Dense CM and CM No-Reinflate stand out for decomposition, with CM No-Reinflate adding stronger execution speed than Dense CM.

 **Insight:** Bitset dominates flat output speed. ROBDD/CUDD dominates symbolic build speed and canonical representation. CM variants dominate representation, decomposition, and operator-level structure. The key insight: ROBDD/CUDD symbolic build and flat truth-table extraction are different tasks with very different performance profiles.

Direct CM equivalence comparison requires both expressions to be normalized to the same ordered CM basis.

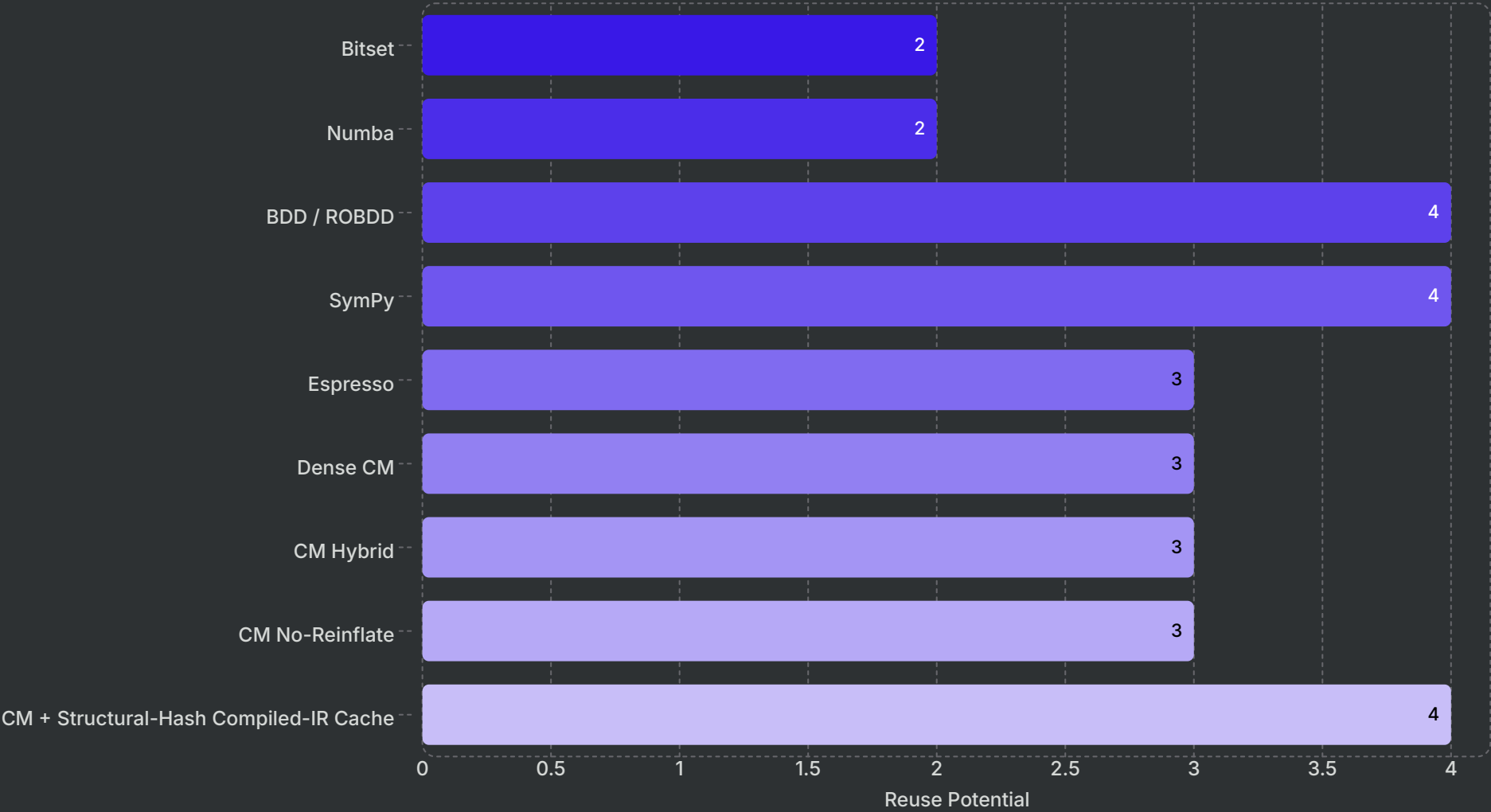
 If the reader only needs a final truth table, bitset is the obvious choice. If the reader needs to inspect, transform, decompose, reuse, or measure the logic that produced the result, CM becomes much more important.

Footnote: ROBDD/CUDD flat output speed is rated 1 because build + truth-table extraction at $n=16$ was 0.513 s (balanced_all_vars) vs 0.000188 s for bitset — a 2727x difference.

Reuse Potential

What happens when the same expression, a structurally similar expression, or a related expression family with shared substructure is evaluated multiple times? Reuse capability is one of CM's key differentiators over flat bitset execution, but CM is not the only reuse-capable representation. ROBDD/CUDD also has strong reuse/canonicalization potential in shared-manager symbolic workflows.

Method



Scale: 1 = Low, 2 = Low-Moderate, 3 = High, 4 = Very High

ROBDD/CUDD also has strong reuse/canonicalization potential, especially in shared-manager symbolic workflows. CM's distinct reuse claim is structural-hash compiled-IR caching and CM-native operator/matrix reuse. Do not overstate CM as the only reuse-capable method.

Reuse capability directly affects total cost of computation in practical workflows:

- Repeated evaluation of the same Boolean function
- Conditioning on different variable assignments
- Compositional builds of larger Boolean circuits
- Related-expression evaluation under partial substitution

The Structural-Hash Compiled-IR Cache results directly support this: once CM compilation is amortized, the total cost gap to bitset narrows substantially.

Structural-hash compiled-IR caching produced 1.51x–1.89x end-to-end speedups for no-reinflate, and cached CM execution reached 2.42x of cached bitset at n=16.



Reuse matters most when the workload is not a one-shot truth-table evaluation. ROBDD/CUDD reuse is strongest for canonical symbolic workflows. CM reuse is strongest for compile-once/evaluate-many workflows with shared CM structure and Structural-Hash Compiled-IR Cache.

Scalability and Failure Modes

Every method has a failure mode. Understanding these is more useful than asking which method is "best" in isolation. Knowing where each approach breaks down allows practitioners to select the right tool for their regime.

Method	Scalability	Predictability	Reusability	Failure Robustness
CM + Structural-Hash Compiled-IR Cache	5	4	5	5
CM No-Reinflate	4	4	4	4
CM Hybrid	4	3	4	3
CM Parallel	3	2	3	3
Dense CM	3	3	3	2
Espresso	3	3	3	3
SymPy	2	3	4	3
ROBDD/CUDD	4	3	5	3
Numba	2	4	2	3
Bitset	2	5	2	3

Scale: 1 = weak, 5 = strong. Higher is better across all columns.

Reusability here means favorable reuse potential. A score of 5 means reuse is strong.

✔

CM-family methods have strong reuse potential. The optimized CM path is most favorable when compiled structure can be reused, dense reinflation is avoided, and related expressions share structure.

📄

Dense CM scaling note: Dense CM's scaling risk is not per-cell complexity — CM cells are binary. The risk is dense matrix materialization: the total number of cells grows exponentially with variable count. The optimized CM path shifts the bottleneck from dense matrix size to IR compilation and execution scaffolding, which can be mitigated through Structural-Hash Compiled-IR Cache and compile-once / evaluate-many workflows.

Large- n feasibility beyond the reported benchmark sizes should be treated as future validation, not already shown.

ROBDD/CUDD note: ROBDD/CUDD is extremely strong for canonical symbolic construction, but remains variable-order sensitive. Dynamic reordering can reduce node count, but it adds overhead. When full truth-table output is required, extraction can dominate and become much slower than bitset or CM no-reinflate. CUDD dynamic reordering with sift was tested at $n=16$: build=0.000274 s, reorder=0.002636 s, extraction=0.912354 s, nodes=68. Reordering reduced nodes but did not change the flat-output conclusion.

📄

CM scalability comes from symbolic IR, no-reinflate output, and Structural-Hash Compiled-IR Cache — not multiprocessing.

APPENDIX

Appendix — Numeric Evidence

Detailed benchmark charts supporting the structural-compilation thesis.

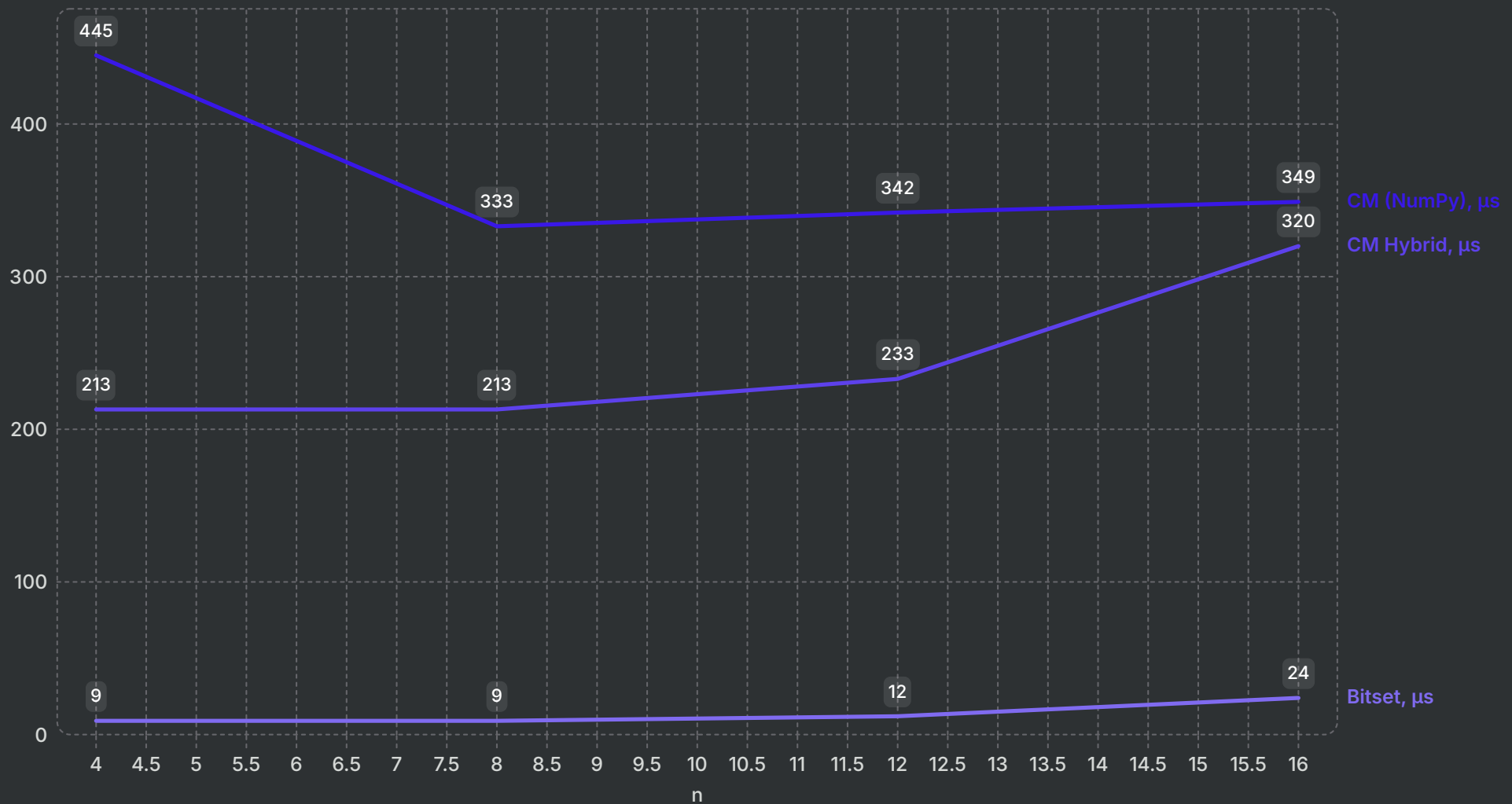
Numeric results summarize the reported CM + bitset experimental results. Rankings are benchmark-regime-specific, especially for random-expression behavior.

Appendix — Section 7 — Numeric Evidence

Hybrid vs. NumPy CM and Bitset

This table shows the first major hybrid comparison. The key finding: hybrid CM improved over NumPy-only CM, but bitset remained faster. Ratios reflect hybrid time relative to the comparison method (values above 1.0 mean hybrid is slower).

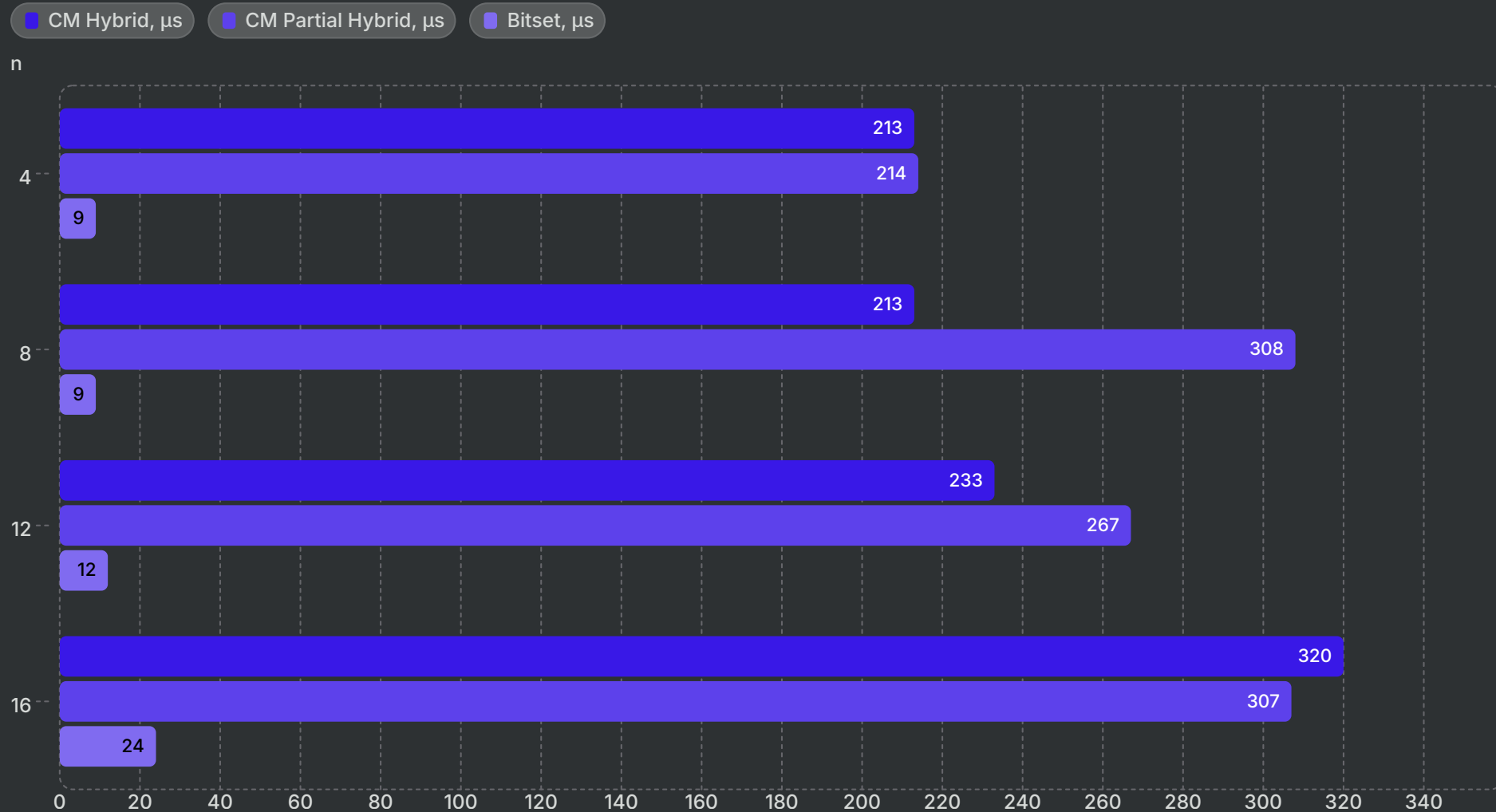
Diagnostics showed hybrid often collapsed to one bitset materialization and zero NumPy materializations, indicating CM reduced the problem before delegating execution. This is pre-no-reinflate/pre-CUDD-context evidence; it shows CM architectural progress but not the final task-specific comparison.



Hybrid improves over NumPy CM, but bitset remains much faster for flat execution. The hybrid/bitset gap narrows from ~31.3x at n=4 to ~8.2x at n=16.

Partial Hybrid vs. Full Hybrid

Partial hybrid preserved more structure, but did not consistently improve runtime.



Useful negative result: Preserving more structure is not automatically faster. In these trials, partial hybrid usually added boundary and parent-combine overhead; a future cost model would be needed to identify cases where preserving intermediate CM structure pays off. This suggests partial hybrid needs a real cost model; threshold-only preservation of root structure was not enough. This is consistent with the broader lesson that preserving structurally similar or related expressions with shared substructure is not automatically faster; a cost model is needed.

Boundary Costs Helped Explain — But Did Not Solve — the Gap

Boundary instrumentation measured the CM/bitset conversion boundary. The boundary was real and reducible, but not the dominant remaining bottleneck.

Finding	Evidence	Interpretation
Boundary overhead was measurable	Bitset conversion, hypercube conversion, alignment, and dispatch were separately timed	The CM/bitset boundary was not free
Optimization helped	Several boundary reductions were around 1.4x–2.0x	Alignment and conversion improvements mattered
Boundary was not dominant	Optimizations did not close the gap to bitset	Dense output and IR/scaffolding costs remained
Next bottleneck identified	Dense final CM reinflation remained expensive	This motivated no-reinflate execution

Two micro-optimizations helped: an alignment fast-path and avoiding a redundant uint8 copy after np.unpackbits.



Key insight: Boundary overhead was real, but not dominant. The next major improvement came from avoiding final dense CM reinflation entirely. Boundary optimization helped but did not solve the main gap; dense output and IR/scaffolding costs remained.

Detailed pre/post boundary timing tables are available in the appendix.

Do Not Compare Symbolic Build to Flat Execution

These are different tasks with different outputs. Comparing them directly produces misleading conclusions.

ROBDD/CUDD build-only answers:

"How quickly can I construct a canonical decision graph? Output: a compact symbolic DAG. Not a flat truth-table vector."

ROBDD/CUDD build + extraction answers:

"How quickly can I produce a full flat truth-table output from an ROBDD? Output: a flat vector. Extraction dominates at n=16 (0.513 s balanced, 0.620 s mixed)."

Bitset answers:

"How quickly can I evaluate all assignments as packed output? Output: a flat truth-table vector. Fastest in all tested regimes (0.000188 s at n=16, balanced)."

CM no-reinflate answers:

"How quickly can CM reduce the structure and return packed/TT output without dense CM matrix reinflation? Output: packed bitset or TT-vector. Best CM flat-output path (0.001688 s at n=16, balanced)."

Dense CM / CM reinflated answers:

"How quickly can I produce the actual dense CM matrix artifact? Output: the full CM matrix. Use only when the matrix itself is the desired output."

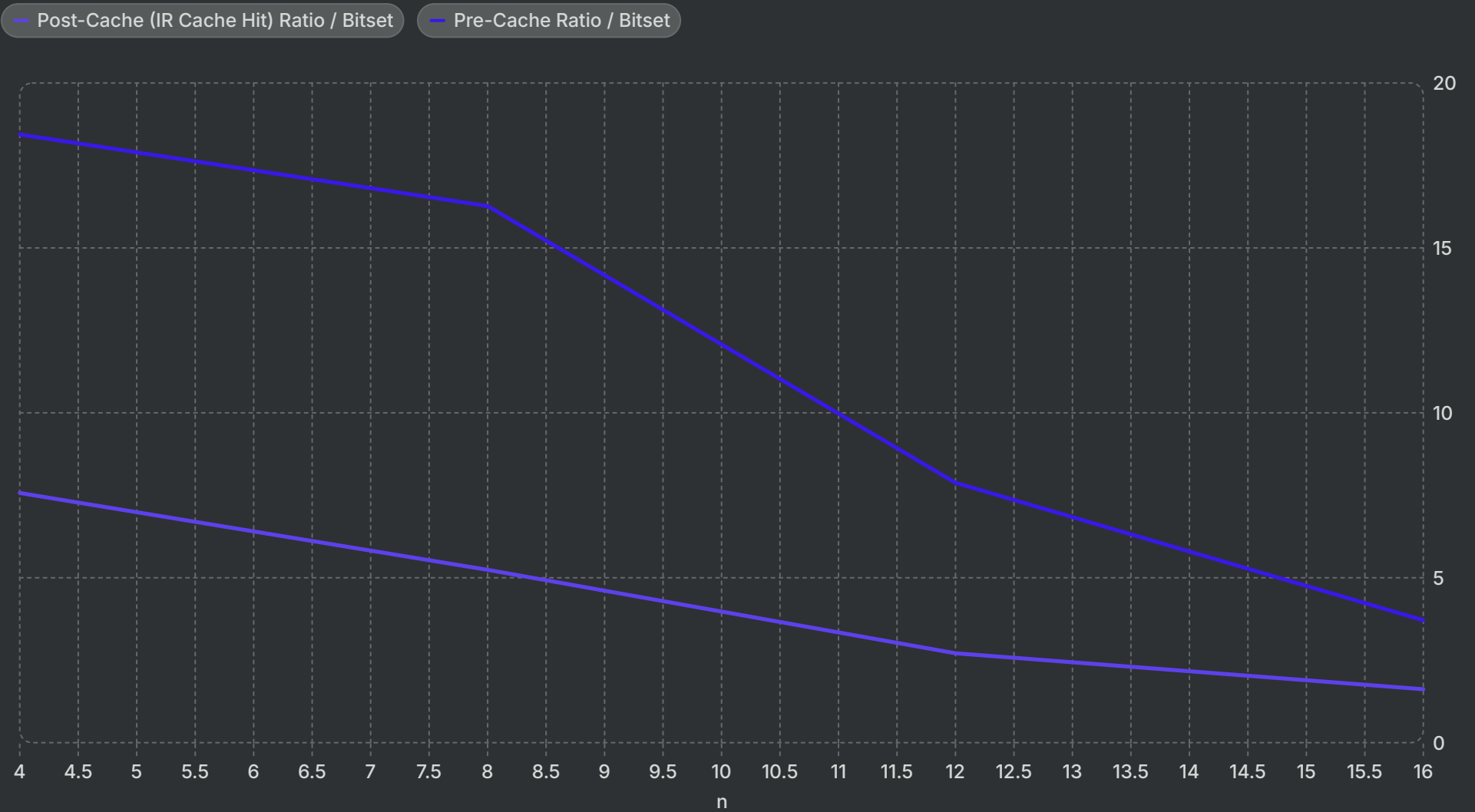
"CUDD/ROBDD is highly competitive, and often strongest, for symbolic graph construction. Bitset wins flat execution. CM no-reinflate is the strongest CM output path when dense CM output is not required."



The CUDD benchmark results confirm this separation clearly: CUDD build-only at n=16 was 0.000138 s (balanced) — near bitset. CUDD build+extract at n=16 was 0.513589 s (balanced) — 2727x slower than bitset. The task determines the winner.

Compiled IR Reuse

Compiled-IR reuse removes the IR compilation cost from repeated evaluations by caching the compiled structure. For repeated or structurally related evaluations, this produces one of the strongest performance results in favor of the CM architecture.



Structural-hash compiled-IR caching dramatically reduces the ratio vs bitset — especially at small n.

Compile-once and cache benefits depend on reuse. They are not expected to help one-shot flat evaluations.

Compare to the pre-cache ratios: at n=4, the gap drops from **18.44x → 7.57x**. At n=16, from **3.71x → 1.62x**. IR compilation was the dominant per-evaluation overhead, not the reduced bitset subproblem execution itself.

The remaining gap at cache-hit time represents execution scaffolding costs: dispatch, result wrapping, variable-order handling, DAG traversal, fixed-variable handling, and bitset invocation overhead.

Compile-once/cache benefits require repeated or structurally related workloads. They do not help unrelated one-shot flat evaluations.

The Compile-Once Insight

Compiled-IR reuse removes the dominant compilation overhead from repeated evaluations — ratio drops from 18.44x to 7.57x at n=4.

CM_parallel: Useful Negative Result, Not the Current Scalability Story

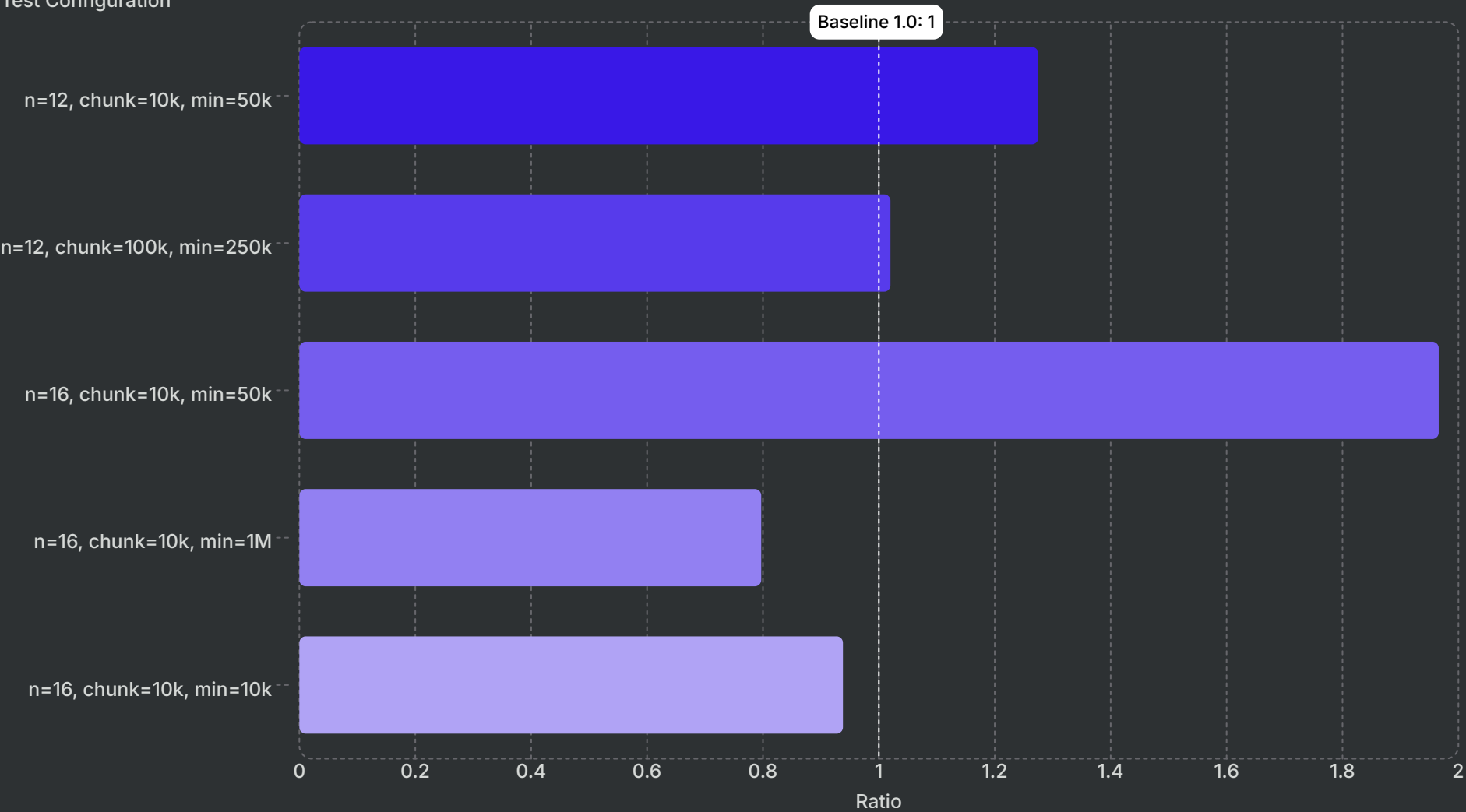
Selected stress results

CM_parallel was tested on the hypothesis that CM operations can involve large matrix/tensor-style work suitable for multiprocessing. However, optimized CM often removed that dense work before multiprocessing could benefit.

In the main validation grid, activation rate was 0.000 for n=8,12,16; parallel_pool_starts stayed at 0 and fallbacks were dominated by small_total_work.

Forced activation showed only intermittent gains: e.g., n=16 with min_work=10000 gave ratio 0.938, a small win, while other settings were neutral or slower.

Test Configuration



CM_parallel is a useful negative result: the results suggest multiprocessing is not the current scalability story. Values below 1.0 = CM_parallel faster; above 1.0 = CM_parallel slower than baseline CM.

CM_parallel remains important as a negative result, but it should not define CM's scalability story. The stronger scalability story is symbolic IR, live-variable reduction, no-reinflation, and structural-hash compiled-IR caching — not multiprocessing.

Recommendation Table: When Another Method Is Best

A practical decision guide for scenarios where a non-CM method is the clearest choice.

Goal	Recommended Method	Why
Fastest one-shot flat Boolean evaluation	Bitset	Strongest raw execution kernel
Fast repeated flat evaluation with no structure needed	Cached bitset	Minimal execution overhead
Mature canonical decision-graph reasoning	ROBDD/CUDD	Best-known optimized baseline for canonical decision graphs, symbolic construction, and equivalence under a fixed variable order. CUDD symbolic build was extremely fast in Docker/Linux runs.
Broad symbolic algebra / readable expression manipulation	SymPy	Mature CAS-style ecosystem for parsing, rewriting, assumptions, simplification utilities, and pretty-printing
Logic minimization	Espresso	Designed for minimization workflows
Full symbolic graph construction	ROBDD/CUDD	CUDD symbolic build was competitive with bitset at n=16 in Docker/Linux runs.
Flat truth-table output	Bitset	Fastest flat execution kernel in all tested regimes.



These methods each solve a different subproblem. Use the method that matches the computation, not the one with the best general reputation.



ROBDD/CUDD build-only is not the same as truth-table output. Include extraction when the desired artifact is a flat output vector. CUDD build+extract at n=16 was 0.513 s vs 0.000188 s for bitset.

Recommendation Table: When CM Is Best

A practical decision guide for scenarios where the CM architecture is the strongest choice.

Goal	Recommended Method	Why
Exact operator / matrix representation	CM / LM; dense CM when the matrix itself is required	The matrix or measurement structure is the desired artifact
Structure-preserving Boolean computation	CM IR / DAG	Preserves decomposition, row/column structure, operator semantics, and reusable substructure
Operator-level decomposition / quotienting / measurement	CM / LM	Native CM operations supported by the original formalism
Compile-once / evaluate-many with structure	CM no-reinflat + structural-hash compiled-IR cache	Amortizes compilation while preserving CM structure
Partial evaluation / conditioning workflows	CM + compiled-IR cache	CM structure can be reused under related variable settings
Future research architecture	CM IR → bitset → no-reinflat → structural-hash compiled-IR cache	Best-supported architecture from the experiments
Avoiding dense output while retaining CM structure	CM no-reinflat	Avoids expensive dense CM matrix reconstruction; returns packed bitset or TT-vector output when appropriate

LM = logical matrix / logical measurement form used in the CM formalism.

Where CMs Help in Implemented Architectures

CMs are most beneficial when the computation is not just "evaluate this Boolean expression once." These scenarios reflect the strongest implementation-supported and experimentally suggested use cases.

Repeated and Related Computations

CM compiled IR can be reused across repeated expressions, structurally similar expressions, or related expression families with shared substructure. Structural-Hash Compiled-IR Cache and compile-once/evaluate-many were among the strongest experimental results. Structural-Hash Compiled-IR Cache produced 1.51x–1.89x end-to-end speedups.

Structured Expressions

CM can exploit repeated subtrees, common subexpressions, and algebraic simplification before execution — often reducing effective problem size before bitset evaluation occurs. Symbolic IR diagnostics reported live variables often around 3–5 and materializations around 7–11.

Partial Evaluation and Conditioning

CM can preserve reusable structure across related variable settings, reducing or avoiding recompilation when cached variants apply.

Avoiding Dense Final Output

No-reinflate avoids forcing results back into dense CM matrices. If the final dense CM matrix is not needed, CM retains its structural role without paying that output cost.



These are implementation-supported or experimentally suggested use cases, not universal wins over Bitset/CUDD. Related expression families and repeated partial contexts are the right next tests.

Where CMs Help in Operator Calculus

Beyond implementation, CM's original formalism supports a matrix-based Boolean operator calculus with capabilities that flat bitsets cannot provide.

CM-Native Symbolic Operator Calculus

Operators can be compared, negated, rotated, transposed, decomposed, and composed as matrices. This is native to the CM formalism and not available in flat bitset execution alone.

Logical Measurement and Reconstruction

CMs / LMs can be measured with ordered bra/ket vectors to recover logical expressions. This makes the expression recoverable from the matrix representation.

Quotienting and Feature Difference

CM quotienting can show what operator content one expression has that another lacks — a form of logical feature difference not native to flat truth-table methods alone.

Interpretability and Analysis

CM exposes decompositions, measurement pathways, and operator relationships that flat bitsets do not. The 16 binary CMs give a compact visual map of logical operators and complement relationships.

❗ This is the formal CM-native value proposition, distinct from benchmark speed. CM exposes operator/matrix structure that supports transformation, measurement, quotienting, reconstruction, and comparison under an ordered basis.

Equivalence Checking: Canonical vs. Output Equivalence

Different methods answer different equivalence questions. These are not the same task.

Bitset	ROBDD/CUDD	CM no-reinflate
Build/evaluate packed outputs and compare integers/vectors. Very fast for small-n full-output equivalence. But it does not preserve symbolic structure.	Build canonical ROBDDs in the same manager/order and compare roots. Equality comparison after construction is essentially free. CUDD equivalence should be benchmarked next in Docker/Linux.	Currently compares output payloads after CM no-reinflate. This is output equivalence, not showed canonical CM structural equivalence.

Native Windows dd.autoref Equivalence, n=16

Rewritten equivalent pairs:	
Bitset total	0.000694 s
ROBDD/dd.autoref total	0.003756 s
CM no-reinflate total	0.006594 s
ROBDD compare per call	~0.185 μs
Near-miss pairs:	
Bitset total	0.000388 s
CM no-reinflate total	0.004921 s
ROBDD/dd.autoref total	0.005188 s
ROBDD compare per call	~0.159 μs

 These are dd.autoref numbers, not CUDD equivalence. Run CUDD equivalence in Docker/Linux. ROBDD comparison after construction is essentially free — this is a signature ROBDD/CUDD strength.

 Current CM no-reinflate equivalence is output-based. Do not claim showed canonical CM structural equivalence unless a canonical CM structural equivalence layer is implemented and shown.

When CM Is Not the Best First Choice

CM is not the right tool for every Boolean task. Use the method that matches the computation.

Scenario	Better Choice	Why
One-shot flat truth-table evaluation	Bitset	CM overhead is not amortized
Very small dense Boolean evaluations	Bitset	Bitset has extremely low absolute overhead
Repeated flat evaluation with no structure needed	Cached bitset	Simplest fast path
Broad general-purpose symbolic algebra / CAS workflows	SymPy	More mature parsing, rewriting, assumptions, simplification utilities, and pretty-printing
Mature canonical decision-graph reasoning	ROBDD/CUDD	Best-known optimized canonical decision-graph baseline; CUDD is the fair optimized baseline, but variable ordering matters.
Logic minimization	Espresso	Designed for minimization / cover-reduction workflows
Multiprocessing expected to save small workloads	Bitset or single-process CM	CM_parallel overhead can exceed useful work after structural reduction
Flat truth-table output	Bitset	Fastest flat execution kernel in all tested regimes.



Important distinction: ROBDD/CUDD, SymPy, and CM solve different symbolic problems. ROBDD/CUDD provides canonical reduced decision graphs under a variable ordering; CUDD is the fair optimized baseline. SymPy provides broad symbolic algebra tooling; CM provides exact operator/matrix representation and CM-native Boolean calculus.

CM equivalence checks require normalization to the same ordered CM basis; BDD equivalence relies on a reduced graph under a fixed variable order.

Symbolic Distinctions: ROBDD/CUDD vs. SymPy vs. CM

These three methods are often compared but solve fundamentally different symbolic problems.

ROBDD/CUDD

Canonical reduced decision graphs under a fixed variable ordering. Strong for symbolic construction and equivalence workflows. CUDD is the fair optimized baseline. Performance remains variable-order sensitive.

SymPy

Broad symbolic algebra environment for parsing, rewriting, assumptions, simplification utilities, pretty-printing, and integration with broader math. Best for CAS-style workflows.

CM

Exact operator/matrix representation. Strong for transformation, decomposition, quotienting, measurement, comparison after basis normalization, and structural compilation.

Important Distinction

These methods solve different symbolic problems. CM should not be described merely as 'structure preserving'; it is a matrix-based Boolean operator calculus. Do not claim CM has canonical equivalence unless a canonical CM structural equivalence layer is implemented and validated. ROBDD/CUDD is the strongest baseline for canonical equivalence.

Appendix — Section 17 (continued)

CM vs. ROBDD/CUDD: Different Strengths

CUDD strengthens the ROBDD side of the comparison. The fair conclusion is not that CM beats ROBDD/CUDD universally — it is that ROBDD/CUDD and CM have different strengths for different tasks.

ROBDD/CUDD	CM
Optimized canonical decision graph under a variable ordering; extremely fast symbolic construction in Docker/Linux CUDD runs	Exact operator / matrix representation under an ordered CM basis
CUDD symbolic build was competitive with bitset at n=16 (0.000138 s vs 0.000188 s, balanced_all_vars)	Preserves operator structure, decomposition, measurement, quotienting, and matrix transformations
Build + truth-table extraction dominated: 0.513589 s at n=16 (balanced_all_vars) — 2727x slower than bitset	Dense CM can be expensive if fully materialized; no-reinflate avoids dense output when packed/TT output is acceptable
Strong for mature equivalence workflows; variable-order sensitive; dynamic reordering can help but adds overhead	Strong for CM-native operator calculus and reusable structural compilation
Variable ordering is central to performance; dynamic reordering with sift was available and used in probe	CM basis normalization is central to equivalence comparison; Structural-Hash Compiled-IR Cache supports compile-once/evaluate-many



CUDD symbolic build is extremely fast: faster than CM no-reinflate and near bitset timing in Docker/Linux runs. But once flat truth-table output is required, extraction dominates. CUDD build+extract remains hundreds to thousands of times slower than bitset and CM no-reinflate.



ROBDD/CUDD is excellent for canonical symbolic construction, symbolic equivalence, and compact representation under favorable variable ordering. CM is a different operator/matrix calculus that becomes useful when structure, measurement, decomposition, quotienting, or no-reinflate execution matters. These are different strengths, not a universal ranking.

Where CMs May Be the Best Method

CMs are strongest when Boolean logic is not merely evaluated, but measured, transformed, decomposed, compared, composed, or reused.

Operator-Level Boolean Calculus

Logical operators have unique CM representations under an ordered variable basis, making operator comparison and transformation direct.

Visual Operator Maps

The 16 binary CMs give a compact visual map of logical operators and complement relationships.

Matrix Transformations

Negation, transposition, and rotation can transform the CM and therefore transform the expression.

Quotienting / Feature Difference

CM quotienting can show how one logical expression differs from another by operator-feature overlap.

Measurement-Based Reconstruction

A CM can be measured by ordered logical bra/ket vectors to recover the corresponding expression.

Composition and Reuse

When operands are aligned, structurally similar or related expressions with shared substructure can be combined by combining their CMs directly; CM IR + Structural-Hash Compiled-IR Cache can reuse compiled structure.

Partial Evaluation / Conditioning

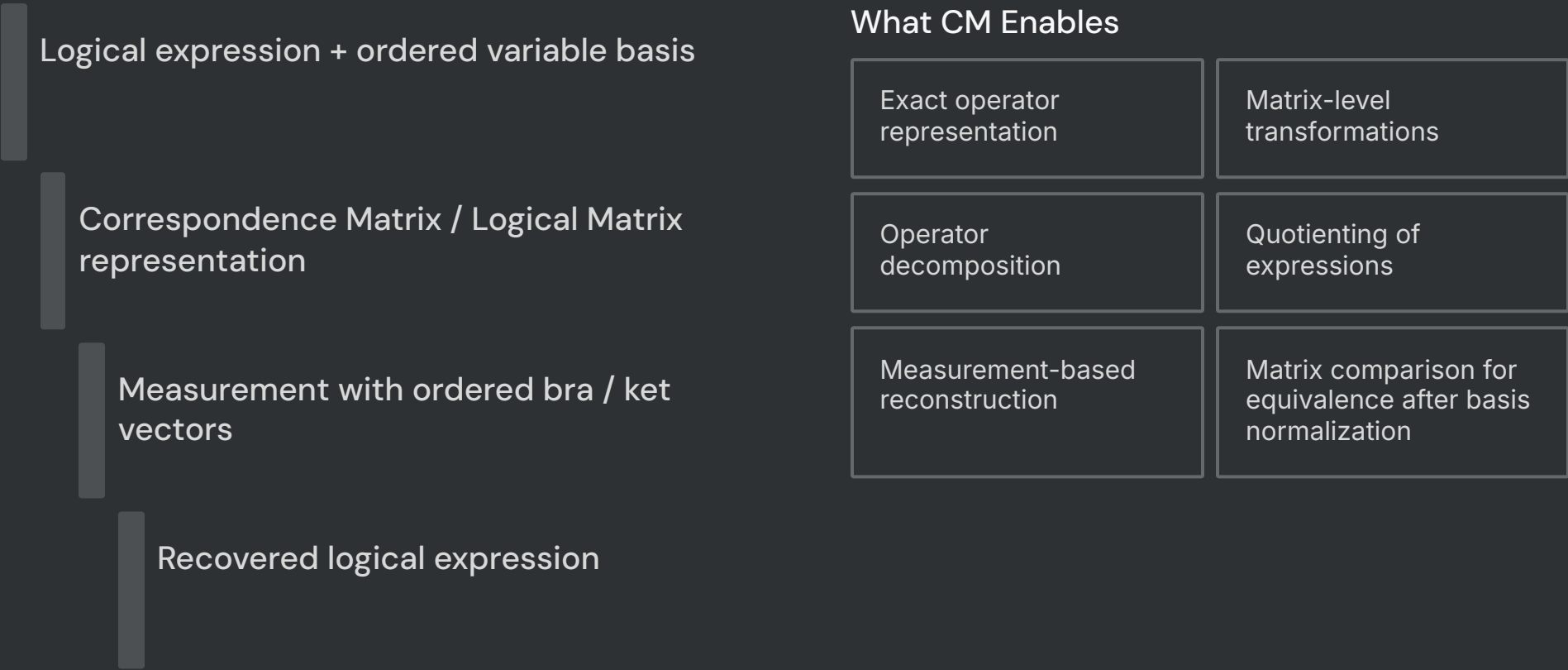
CM IR can preserve reusable structure across related variable assignments, especially when compiled-IR cache or structural-hash cache is used.



Strongest CM-native claim: CMs may be the best go-to method when Boolean logic is treated as a matrix-measurable operator system rather than only a truth table. 'May be best' is the correct wording — not 'is best' unless benchmarked. Best when Boolean logic is being measured, transformed, decomposed, compared structurally, composed, or reused — not merely evaluated once.

CMs and Exact Logical Representation

Why CM representation is stronger than simple 'structure preservation'



```
Expression A → normalize to basis [X,Y,Z] → CM_A
Expression B → normalize to basis [X,Y,Z] → CM_B
CM_A == CM_B ⇒ equivalent under that basis
```

This is a matrix-basis equivalence statement, not a claim that the current implementation has a proven canonical CM equivalence engine comparable to ROBDD/CUDD.

- ⚠ Equivalence caveat: CM equivalence may be checked by matrix comparison after correct basis normalization; canonical CM structural equivalence remains a future formal/implementation target. This is a matrix-basis equivalence statement, not a claim that the current implementation has a showed canonical CM equivalence engine comparable to ROBDD/CUDD.**
- i The original CM formalism is stronger than a generic graph or tree representation. Under an ordered variable basis, logical operators correspond to unique correspondence matrices, and logical expressions can be recovered through ordered bra/ket measurement. This makes CM an operator/matrix measurement formalism, distinct from BDD decision graphs and SymPy expression trees.**

Which Method Wins Where?

The best method depends on what the computation is trying to optimize: speed, canonical equivalence, minimization, general algebra, or CM-native operator calculus.

Task	Best Method	Caveat
Flat one-shot truth-table execution	Bitset	Fastest packed output in tested regimes.
Flat output with CM structural preprocessing	CM no-reinflate	Best CM path when dense CM output is not required.
Repeated CM-structured evaluation	CM no-reinflate + Structural-Hash Compiled-IR Cache	Best when structure recurs and compile cost can be amortized.
Canonical symbolic decision-graph construction	ROBDD/CUDD	CUDD symbolic build was extremely fast in Docker/Linux runs (0.000138 s at n=16, balanced_all_vars).
Canonical equivalence workflows	ROBDD/CUDD	Benchmark this explicitly; canonical ROBDD equivalence is a signature strength. Not yet benchmarked in this project.
Exact operator/matrix calculus	CM	Best for CM-native transformation, decomposition, measurement, quotienting, and reconstruction.
Dense CM matrix artifact	Dense CM / CM reinflated output	Use only when the matrix itself is the desired output.
Broad symbolic algebra tooling	SymPy	General CAS-style ecosystem.
Logic minimization	Espresso	Specialized cover minimization.

Structural-Hash Compiled-IR Caching reuses previously compiled expression structures when the same structural graph or shared structural subgraphs reappear. This avoids repeated IR construction and supports the CM-as-compiler architecture.

**ROBDD/CUDD build-only is not the same as truth-table output.** Include extraction when the desired artifact is a flat output vector. In the tested regime, CUDD build+extract at n=16 was 0.513 s (balanced_all_vars) vs 0.000188 s for bitset — a 2727x difference.

**Representation vs. Execution:** Bitset wins flat execution. ROBDD/CUDD wins canonical symbolic construction and mature decision-graph reasoning. SymPy wins broad symbolic algebra tooling. CM wins exact Boolean operator representation and CM-native operator calculus. These methods are not competing to solve the exact same problem.

Final Interpretation

Bitset is the right flat execution kernel. ROBDD/CUDD is the mature optimized canonical decision-graph baseline. CM is the right Boolean operator calculus and structural compiler.

The graph sequence shows a progression: dense CM is not the execution winner; hybrid CM improves over NumPy CM; no-reinflate removes dense output cost; Structural-Hash Compiled-IR Caching makes CM more competitive when structure repeats. CUDD results strengthen the comparison: optimized ROBDD construction is extremely fast, but flat truth-table extraction remains a different task and remains much slower than bitset and CM no-reinflate in the tested regime.

A skeptical reader may look at the raw speed tables and conclude that bitset wins — and that is true for one-shot flat evaluation. But the broader tables reveal why CM still matters and where it belongs in a production Boolean computation architecture.

- **CM preserves exact operator structure**
Matrix-measurable logical representation: transformable, decomposable, quotientable, comparable, composable, and reconstructable.
- **CM avoids dense output through no-reinflation**
Skipping dense CM matrix reconstruction was one of the strongest implementation improvements.
- **CM becomes much more competitive when compile cost is amortized**
At $n=16$, full hybrid was $\sim 8.2x$ slower than bitset; cached CM was $2.42x$ slower than cached bitset in the repeated-evaluation setting.
- **CM is a Boolean operator calculus, not merely a structure-preserving layer**
Logical operators can be visualized as matrices, transformed, quotiented, decomposed, measured by ordered bra/ket vectors, and composed.

⚠ **Caveat:** The current data supports a narrowing gap through $n=16$, especially for no-reinflate and cached execution. Larger- n validation is needed before claiming true asymptotic convergence.

① **Final thesis:** Bitset is the flat-output execution kernel. ROBDD/CUDD is the mature optimized canonical decision-graph baseline. CM is an exact Boolean operator/matrix calculus and structural compiler. The strongest CM implementation path is CM IR → structural reduction → bitset execution → no-reinflate → Structural-Hash Compiled-IR Cache.

CM represents and reduces. Bitset executes. ROBDD/CUDD canonicalizes under favorable ordering. SymPy manipulates broad symbolic expressions. Espresso minimizes.

The best current thesis is not 'CM replaces bitset.' It is: CM represents, reduces, and reuses Boolean structure; bitset executes the reduced subproblem.

Where Structural Boolean Computing Could Be Valuable

CM may be strongest wherever Boolean logic must be inspected, transformed, reused, or routed — not merely evaluated once.

Verification & Hardware

EDA and formal verification — structural equivalence, decomposition, operator comparison

Hardware synthesis — reusable logic structure, partial evaluation

Software & Systems

Compiler optimization — Boolean IR as a structural layer

Security policy analysis — access-control logic decomposition

Configuration & feature-flag dependency analysis — structural reuse

AI & Symbolic Reasoning

Neuro-symbolic AI — Boolean structure as a reasoning substrate

Knowledge graph rule evaluation — structural logic reuse

Automated rule auditing — operator-level comparison

Science & Biology

Computational biology Boolean networks — operator-level analysis

Digital twins — reusable logic structure across related scenarios



CM may be valuable anywhere Boolean structure must be inspected, transformed, reused, or routed. The common thread: the workload is not one-shot flat evaluation.



Caveat: These are potential application areas, not proven deployments. The next step is validating CM's structural advantages in real structured workloads.

Technology Readiness: What Is Validated vs. What Is Next

The deck is stronger when it distinguishes proven results, prototype architecture, and research roadmap.

Validated in Current Experiments	Prototype / Partially Developed	Research / Future Work
No-reinflat improves CM output path	CM IR / DAG structural representation	Canonical CM structural equivalence
Structural-hash cache improves repeated evaluations	Structural reduction and live-variable analysis	Automatic backend routing cost model
IR compilation cost can be amortized	Basis normalization scaffolding	Quotienting workflows
Bitset remains the flat-output baseline	Backend delegation to Bitset	Larger-n validation
CUDD is extremely strong for symbolic build	Reuse detection via structural hash	Real-world workload validation
CM_parallel is not the current scalability story		Integration with CUDD/SAT/Esspresso/SymPy

③ Separating these three levels is not a weakness — it is what makes the project credible to expert audiences.

Beachhead Market Candidates

CM is broad, but the first wedge should be chosen where structure and reuse matter most.

Beachhead	Why CM Fits	Main Risk	Maturity
Formal Verification / EDA	Structure, equivalence, symbolic workflows; CUDD is already used here	Mature incumbents (CUDD, ABC, Cadence)	Research
Rule / Configuration Systems	Repeated related logic, explainability, structural reuse	Need real customer datasets	Prototype
Computational Biology Boolean Networks	Operator-level analysis, interventions, pathway logic	Domain validation required	Research
Neuro-Symbolic AI	Structural reasoning, reuse, operator-level Boolean logic	Emerging market, unclear integration path	Research
Compiler / Program Analysis	IR mindset, optimization passes, structural reuse	Integration burden with existing toolchains	Research



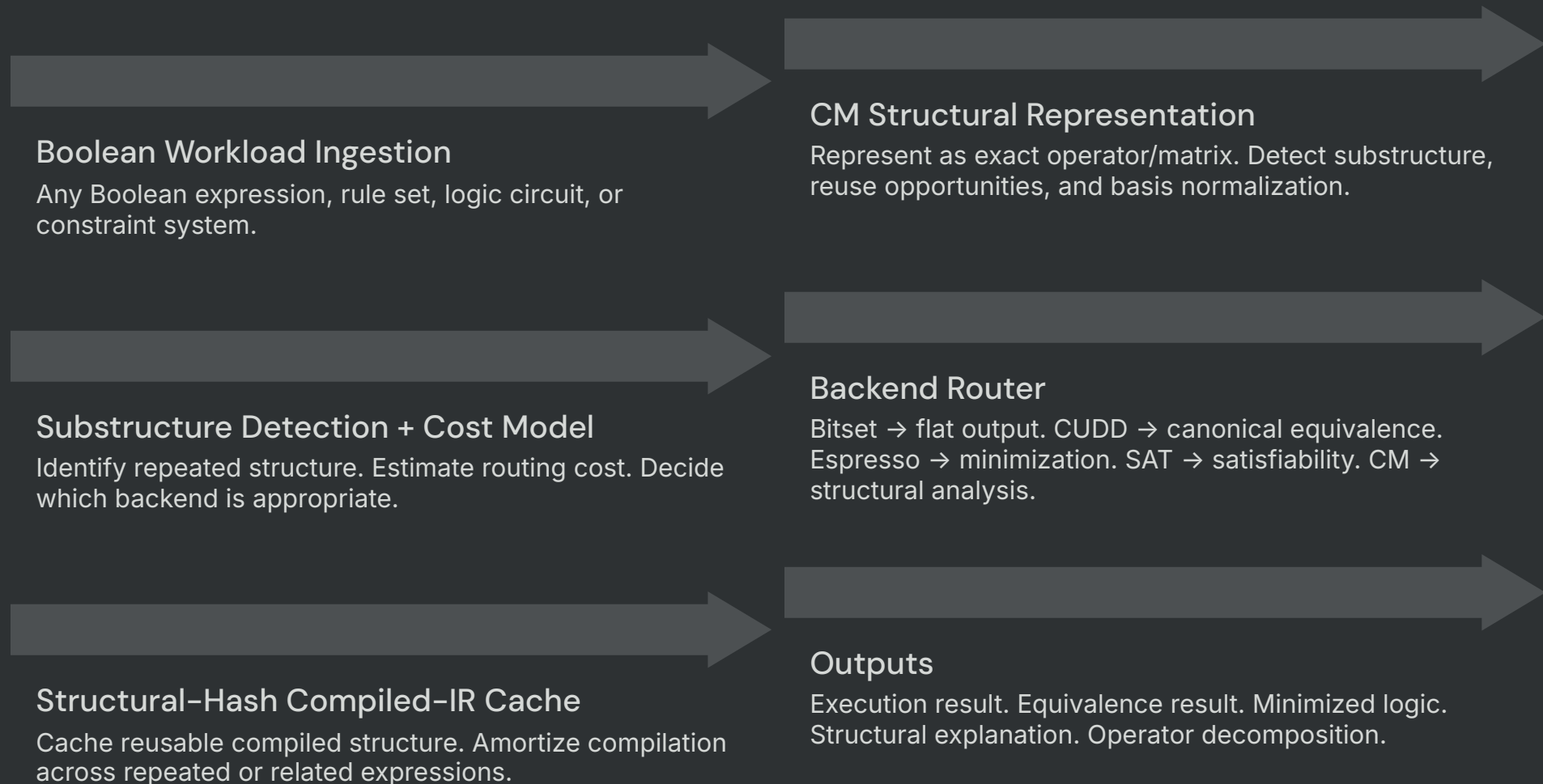
The strongest beachhead is where Boolean structure is already valued — formal verification and EDA — because CUDD is already the baseline and CM's structural layer is a natural complement.



Caveat: These are candidate markets, not validated deployments. Customer discovery and real-workload validation are required before committing to a beachhead.

The Product Vision: A Structural Intelligence Layer for Boolean Computation

Not a faster evaluator — a Boolean computation router and compiler.



The product is not 'faster bitset.' The product is a Boolean computation router/compiler that makes the existing toolbox smarter.



This is a research prototype vision. The current implementation validates the CM structural layer and no-reinflate/cache architecture. Backend routing and cost model are future work.

Why This Could Become a Platform

Large Addressable Problem

Boolean logic appears across AI, EDA, compilers, security, biology, and configuration systems. The substrate is universal.

Fragmented Toolbox

Existing tools are strong but specialized. No structural layer currently coordinates them. CM may provide that layer.

Defensible Technical Differentiation

CM's operator/matrix formalism is formally grounded. The structural-hash compiled-IR cache, no-reinflat architecture, and backend routing are implementable and testable.

Foundation (Now)

Validate CM structural layer. Benchmark CUDD canonical equivalence in Docker/Linux. Stabilize no-reinflat and structural-hash cache.

Routing (Medium-term)

Build backend cost model. Add SAT/CUDD/Esspresso integration. Automatic engine selection.

Benchmarks (Near-term)

Test real structured workloads. Validate larger-n scaling. Build repeated/related workload corpus.

Productization (Long-term)

Developer API. Visualization tools. Domain integrations. Structural explanation layer.

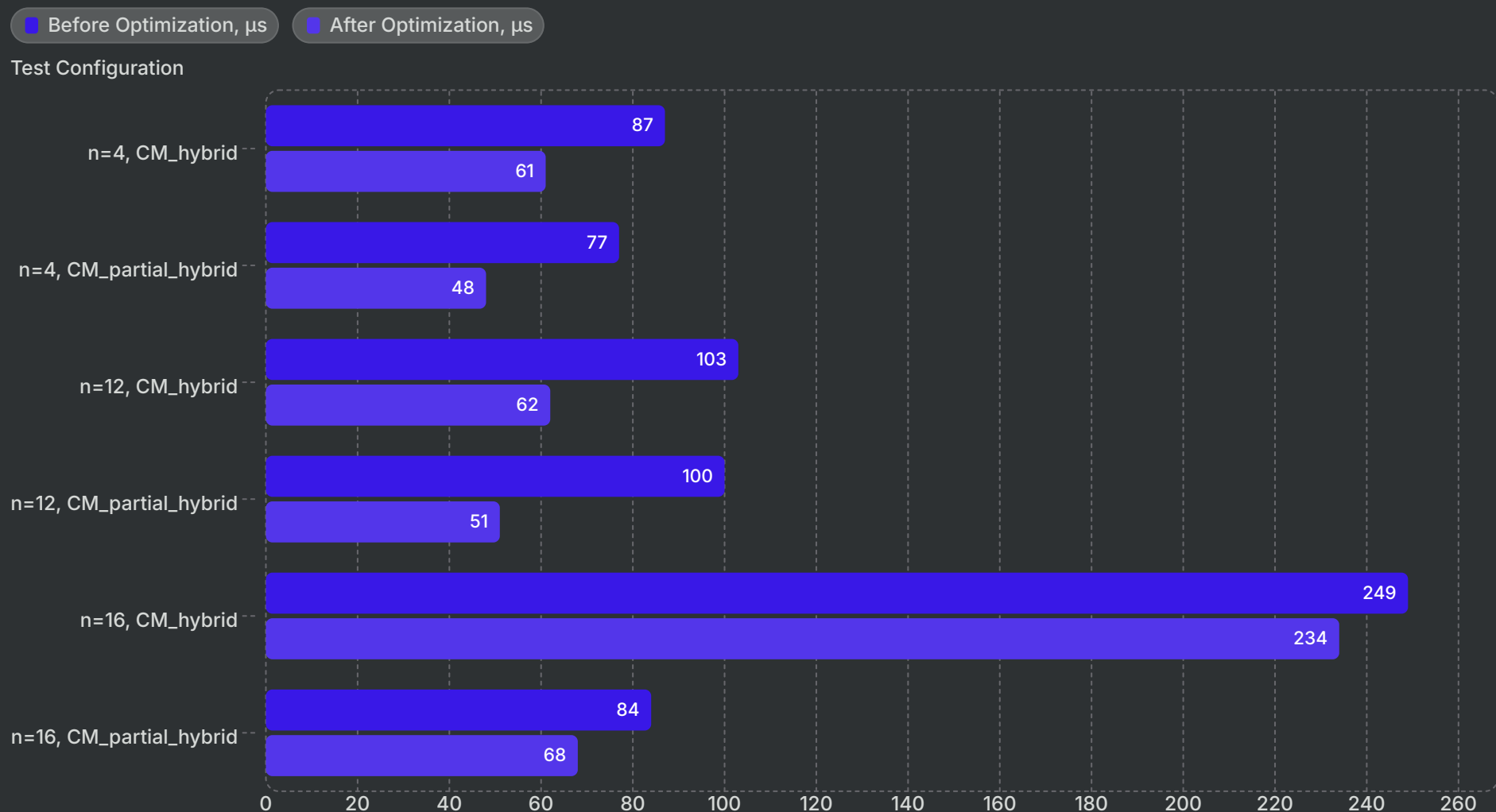
CM is not the fastest hammer. It is the layer that helps decide which hammer to use, what structure to preserve, and what can be reused.

Appendix A

Detailed Boundary Timing: Pre- and Post-Optimization

Full pre- and post-optimization boundary timing tables from Section 9. These were moved from the main slide to reduce information density.

Boundary optimizations included an alignment fast-path and removing redundant uint8 copying. These reduced boundary costs but did not close the bitset gap.



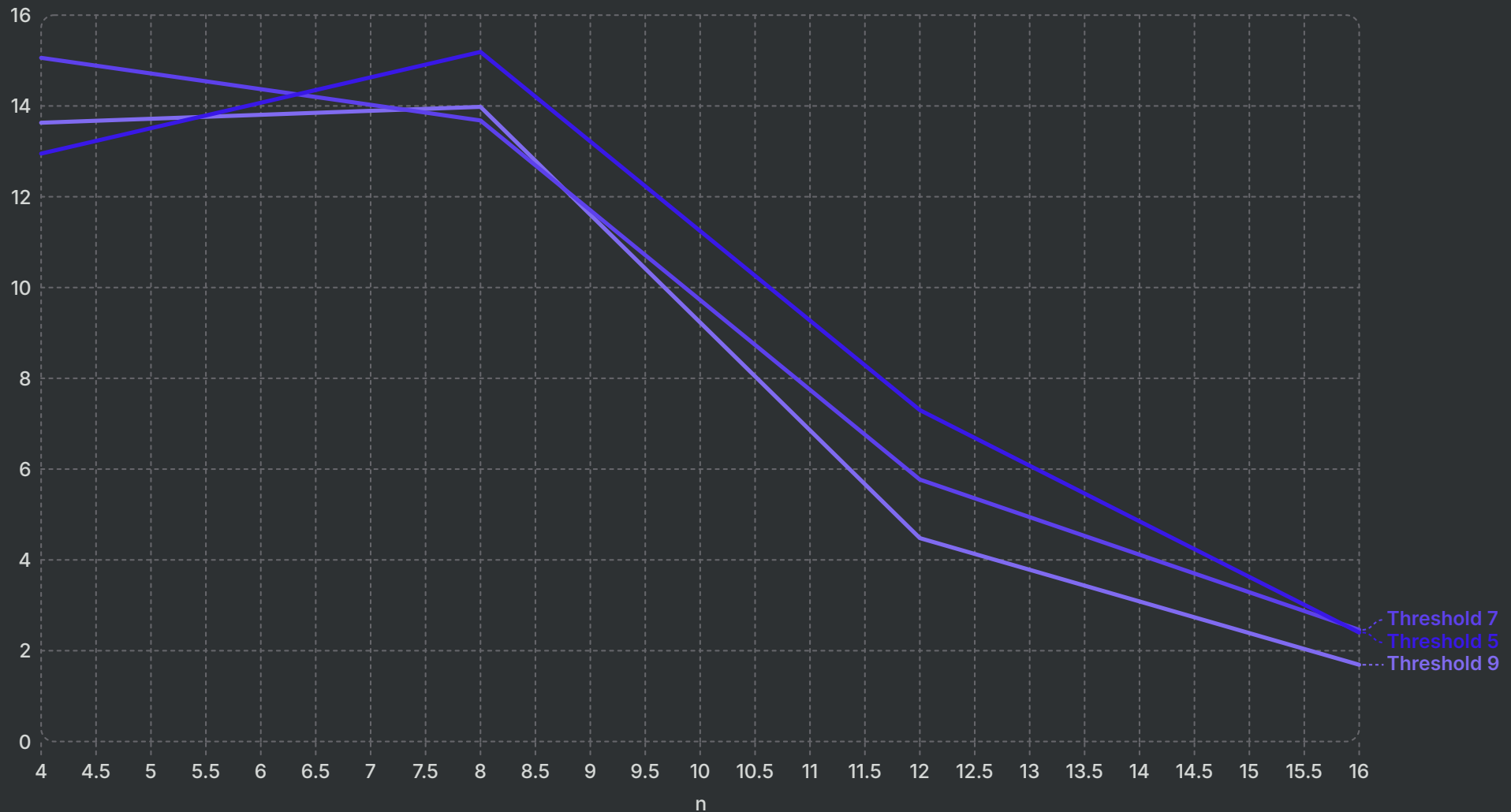
Boundary optimizations helped, especially at smaller sizes (1.4x–2.0x reduction), but did not close the bitset gap.

Appendix B

No-Reinflate Threshold Comparisons

Full threshold comparison tables from Section 10. Threshold = 5 results are shown here alongside Threshold = 9 for completeness.

Threshold Comparison: NR / Bitset Ratio



Lower is better. All thresholds confirm no-reinflate consistently outperforms dense CM.

Threshold 9 produced the strongest (lowest) n=16 ratio (1.69x) in these reported runs. Threshold 5: 2.40x, Threshold 7: 2.46x at n=16.

Compare to Threshold = 9 (shown in Section 10): at n=16, Threshold = 9 reaches 1.69x vs. Bitset, while Threshold = 5 reaches 2.40x. Threshold = 9 is generally the stronger result.

In these reported runs, threshold 9 produced the strongest n=16 ratio to bitset, but threshold 5 and 7 also confirmed the same qualitative finding: no-reinflate was consistently faster than dense CM and usually faster than ordinary hybrid.

Appendix C

Benchmark Scope and Caveats

Important limitations and scope boundaries for interpreting all benchmark results in this deck.

Benchmark Sizes

Benchmark sizes were primarily $n=4, 8, 12, 16$. Larger- n validation remains future work and should not be assumed from these results.

ROBDD/CUDD Scope

ROBDD/CUDD is extremely strong for symbolic construction and canonical reasoning, but variable ordering matters. CUDD build-only is not flat output. Build + truth-table extraction must be included when comparing against Bitset or CM no-reinflate for flat output.

Bitset Comparison Scope

Bitset comparisons are strongest for flat truth-table evaluation. CM advantages are strongest when structure, reuse, measurement, or composition matters.

CM Scalability Claims

CM scalability improvements come from symbolic IR, live-variable reduction, no-reinflate output, and structural-hash compiled-IR caching. Large- n feasibility beyond the reported benchmark sizes should be treated as future validation.

CM_parallel

CM_parallel is not currently a strong selling point. Structural reduction often leaves too little dense work for multiprocessing to pay off. This is a useful negative result, not a scalability story.

Cross-Method Timing Evidence

The most detailed numeric tables in this deck focus on CM variants vs bitset. Cross-method rankings involving BDD, SymPy, Espresso, and Numba should be treated as benchmark-regime summaries unless the full cross-method timing table is included.

CUDD Environment

CUDD results were produced in Docker/Linux/WSL2 using dd.cudd. Native Windows used dd.autoref because dd.cudd was not importable. Cross-environment comparisons should be interpreted with care.

CM Equivalence

Current CM no-reinflate equivalence is output-based. Do not claim proven canonical CM equivalence unless a canonical CM structural equivalence layer is implemented and proven. ROBDD/CUDD is the strongest baseline for canonical equivalence checking.

Appendix D

Glossary

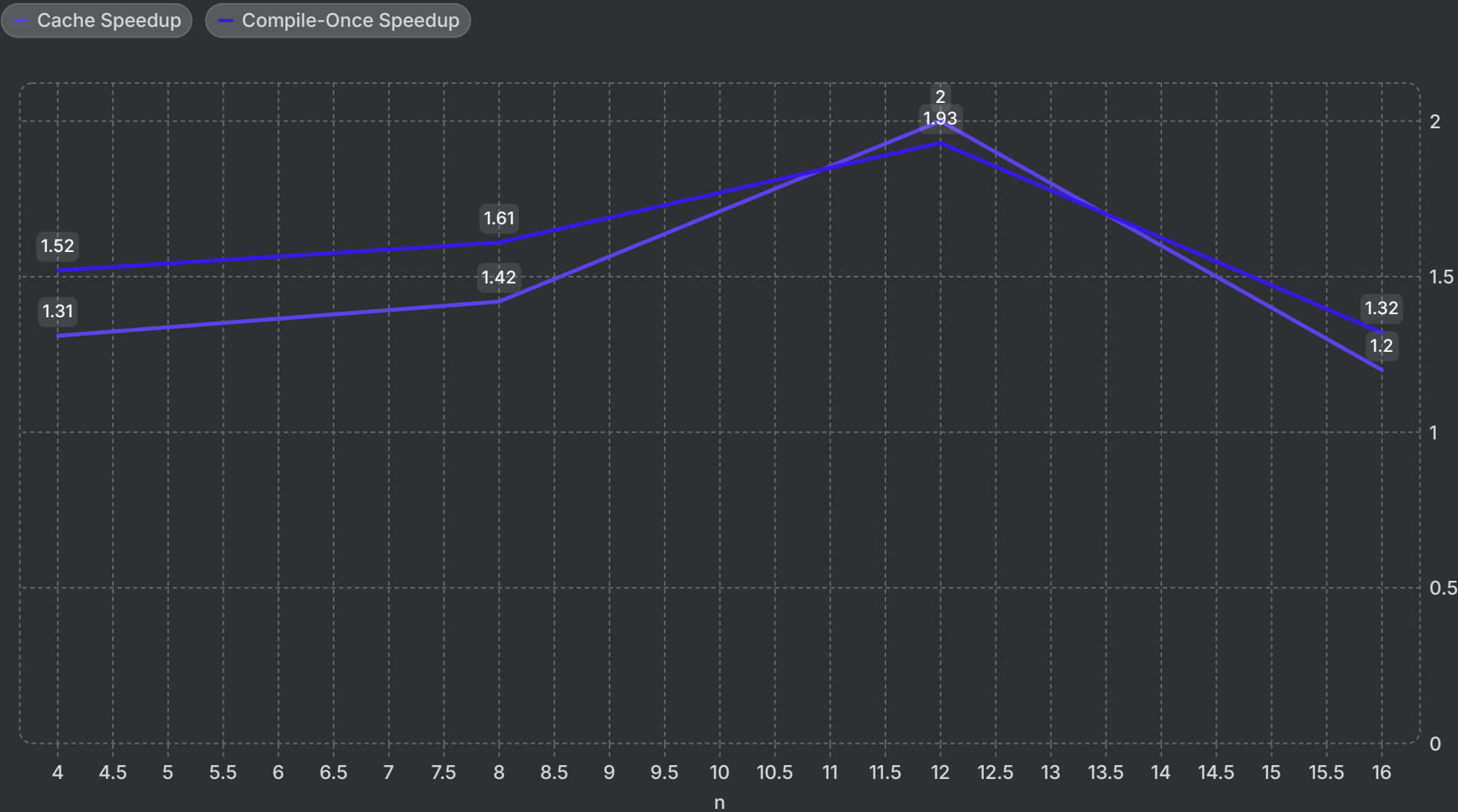
Key terms used throughout this deck.

CM	Correspondence Matrix. A binary matrix representation of a Boolean logical operator under an ordered variable basis.
LM	Logical Matrix / Logical Measurement form. Used in the CM formalism for measurement-based expression recovery.
No-reinflate	An optimization that avoids converting the final CM computation result back into a dense 2D CM matrix unless that matrix is explicitly required.
Bitset	A packed binary array used for fast flat truth-table evaluation. Represents truth assignments compactly and evaluates many assignments at once using bitwise operations.
ROBDD/CUDD	Reduced Ordered Binary Decision Diagram. Canonical decision-graph representation under a fixed variable ordering. CUDD is the optimized C library baseline; dd.autoref is the pure-Python baseline.
IR	Intermediate Representation. The compiled symbolic structure used by CM to represent and execute Boolean expressions without dense matrix materialization.
Structural-Hash Compiled-IR Cache	A cache that stores compiled CM IR structures keyed by structural hash. When an equivalent expression graph is encountered again, the previously compiled IR can be reused instead of being rebuilt. This supports compile-once / evaluate-many workflows and repeated evaluation of structurally identical expressions.
Structural hash	A hash of the CM expression's structural form, used to identify equivalent expressions for cache lookup.
Ordered CM basis	The ordered variable basis under which a CM is defined. Equivalence comparison between CMs requires both to be normalized to the same ordered basis.
Bra / ket measurement	A measurement operation using ordered logical bra and ket vectors to recover a logical expression from its CM / LM representation.
CM no-reinflate	CM execution mode that uses CM IR reduction and bitset execution while avoiding final dense 2D CM matrix reconstruction.
Representation code 2	Diagnostic code indicating packed bitset output in the no-reinflate path.
ROBDD/CUDD	Reduced Ordered Binary Decision Diagram implemented via the CUDD library. Canonical decision-graph representation under a fixed variable ordering. CUDD symbolic build is extremely fast; build+truth-table extraction is a different and much slower task.
CUDD build-only	Symbolic graph construction only — produces a compact ROBDD DAG, not a flat truth-table output vector.
CUDD build+TT extraction	Symbolic build plus full truth-table output generation. Extraction dominates at n=16 (0.513 s balanced_all_vars vs 0.000188 s for Bitset).
Dense CM	Full CM matrix artifact. Use only when the matrix itself is the required output.

IR Reuse and Structural-Hash Cache Details

Compile-once and Structural-Hash Compiled-IR Cache wall-time results.

Compile-Once / Cache Wall-Time Results



Compile-once and structural-hash caching both support the CM-as-compiler architecture, with peak gains around $n=12$ in these runs.

What the Graph Means

The graph shows that compile-once and structural-hash caching reduce repeated work. The benefit is strongest when expressions or structurally identical expressions recur.

This supports the CM-as-compiler framing: CM compilation should be amortized across repeated or related evaluations, rather than paid again for every one-shot flat evaluation.

❏ Interpretation: caching does not make CM the fastest one-shot evaluator. It makes CM more competitive when structure is reused — while preserving representation, decomposition, and conditioning advantages that flat bitset execution does not provide.

❏ "Structural-hash compiled-IR caching supports the CM-as-compiler framing by allowing previously compiled expression structures to be reused when structurally identical expression graphs reappear."

CUDD / ROBDD Benchmark Environment and Caveats

CUDD results require careful interpretation due to environment differences and the build-vs-extraction distinction.

Environment

- Docker Desktop on WSL2; image: python:3.10-slim; Python 3.10.20; dd version 0.5.7
- dd.cudd import succeeded in Docker/Linux; dd.autoref import succeeded
- dd.sylvan and dd.buddy did not import
- Native Windows installed the pure-Python dd wheel and could not import dd.cudd
- CUDD results are Docker/Linux results only; dd.autoref results are from native Windows unless otherwise noted
- Cross-environment comparisons (CUDD vs. dd.autoref) should be interpreted with care

Dynamic Reordering Probe, n=16 (fixed-order + sift)

Build time	0.000274 s
Reorder time (sift)	0.002636 s
Build + reorder	0.002964 s
Extraction time	0.912354 s
Build + extract (total)	0.916939 s
Nodes after reordering	68



Dynamic reordering with sift was available and used. It reduced nodes in the probe (68 nodes vs 162 in the fixed-order balanced_all_vars run), but added reordering overhead (0.002636 s) and did not improve flat-output extraction enough to change the conclusion. Do not directly compare the fixed-order + sift probe to best-of-k order sweeps as if they are the same experiment.

Key Caveats

- CUDD build-only and build+extract must be reported separately; they are different tasks
- Extraction was measured only up to n=16; larger-n extraction behavior is not yet characterized
- The extraction implementation evaluates assignments through the ROBDD; if future optimized bulk extraction is added, update those numbers
- CUDD dynamic reordering was available and used in the probe; results are diagnostic, not a main performance claim
- ROBDD/CUDD is the strongest baseline for canonical equivalence checking; this has not yet been benchmarked in this project

Next Experiments: Where CM's Practical Value Should Be Tested

The current benchmarks establish baselines. These experiments would test CM's strongest practical claims.

 First priority: Run CUDD equivalence benchmark in Docker/Linux. Current equivalence numbers are dd.autoref on native Windows, not CUDD.



Experiment A — "Related Expression Families"

Question: Does CM no-reinflate + Structural-Hash Compiled-IR Cache save total workload time when many expressions share substructure?

Success would look like: CM compiled-IR cache substantially faster than CM no-cache; CM approaches or beats Bitset total workload time for structurally related families; cache hits show real reuse.



Experiment B — "Partial Evaluation Contexts"

Question: Can CM reuse compiled structure and reduce live-variable work across many fixed-variable contexts?

Success would look like: CM cached partial evaluation substantially faster than CM no-cache; live variables shrink; total time becomes competitive over many contexts.



Experiment C — "Operator Difference / Quotient Workload"

Question: Can CM compute useful structural/operator differences that Bitset and ROBDD do not natively expose?

Success would look like: CM structural delta provides a distinct computational output; detects structural differences even when semantic truth delta is zero; avoids full truth-table expansion in useful cases.

 These experiments would test whether CM's structural advantages translate to practical workload wins. Until then, the strongest claim remains: CM represents, reduces, and reuses Boolean operator structure; Bitset executes flat output; ROBDD/CUDD canonicalizes symbolic decision graphs.

Appendix F

What Worked vs. What Did Not

A summary of the experimental sequence: positive results and important negative results.

High Impact / Successful

- Bitset backend — strongest flat baseline
- Symbolic CM IR / DAG — live-var reduction
- No-Reinflate — major output cost reduction
- IR Reuse / Structural-Hash Compiled-IR Cache — 1.51x–1.89x speedups
- CUDD as fair ROBDD baseline

Moderate Impact / Successful

- Balanced layout — ~1.3x–1.8x improvements
- Full Hybrid CM — improved over NumPy CM

Interesting but Limited

- Boundary micro-optimizations — helped but not dominant
- Partial hybrid — preserved structure, inconsistent runtime

Negative Results / Not Main Path

- CM_parallel — activation often zero, gains intermittent
- Dense CM as raw evaluator — not fastest

Did Not Work as Headline

- Universal speed ranking
- Dense CM as raw executor
- CM_parallel as main scalability story
- Partial hybrid without cost model
- Comparing CUDD build-only to Bitset flat output

"The strongest claim is not that CM replaces Bitset or CUDD. The strongest claim is that CM adds an exact operator/matrix layer for representing, transforming, reducing, and reusing Boolean structure — then delegates flat execution to Bitset when flat output is the desired artifact."